

01
02
03
04 **P3109™/D1**
05
06 **Draft Standard for**
07
08 **Arithmetic Formats for Machine Learning**

09 Developed by the

10
11 **Microprocessor Standards Committee**
12 of the
13 **IEEE Computer Society**
14

15 Approved (Date Approved)
16

17
18 Copyright © 2026 by The Institute of Electrical and Electronics Engineers, Incorporated
19 Three Park Avenue
20 New York, New York 10016-5997, USA

21 All rights reserved.
22

23 This document is an unapproved draft of a proposed IEEE Standard. As such, this document is subject to change.
24 USE AT YOUR OWN RISK! IEEE copyright statements SHALL NOT BE REMOVED from draft or approved IEEE
25 standards, or modified in any way. Because this is an unapproved draft, this document must not be utilized for any con-
26 formance/compliance purposes. Permission is hereby granted for officers from each IEEE Standards Working Group
27 or Committee to reproduce the draft document developed by that Working Group for purposes of international stan-
28 dardization consideration. IEEE Standards Department must be informed of the submission for consideration prior
29 to any reproduction for international standardization consideration (stds-ipr@ieee.org). Prior to adoption of this
30 document, in whole or in part, by another standards development organization, permission must first be obtained from
31 the IEEE Standards Department (stds-ipr@ieee.org). When requesting permission, IEEE Standards Department
32 will require a copy of the standard development organization's document highlighting the use of IEEE content. Other
33 entities seeking permission to reproduce this document, in whole or in part, must also obtain permission from the IEEE
34 Standards Department.

35 IEEE Standards Department
36 445 Hoes Lane
37 Piscataway, NJ 08854, USA
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52

Abstract: This standard defines binary arithmetic and data formats optimized for machine learning domains. Compact binary floating-point interchange formats are specified, parameterized by bitwidth, precision, signedness, and domain. Operations including rounding with saturation modes and conversion between P3109 formats and IEEE Std 754™ formats are defined. A consistent and flexible arithmetic framework is provided for machine learning systems in hardware and software implementations.

Keywords: floating-point formats, floating-point operations, 3109

The Institute of Electrical and Electronics Engineers, Inc.
3 Park Avenue, New York, NY 10016-5997, USA

Copyright © 2026 by The Institute of Electrical and Electronics Engineers, Inc.
All rights reserved. Published (Date Published). Printed in the United States of America.

IEEE is a registered trademark in the U.S. Patent & Trademark Office, owned by The Institute of Electrical and Electronics Engineers, Incorporated.

PDF: ISBN 978-0-XXXX-XXXX-X STDXXXXX
Print: ISBN 978-0-XXXX-XXXX-X STDPDXXXXX

IEEE prohibits discrimination, harassment, and bullying.

For more information, visit <https://www.ieee.org/about/corporate/governance/p9-26.html>.

No part of this publication may be reproduced in any form, in an electronic retrieval system or otherwise, without the prior written permission of the publisher.

Important Notices and Disclaimers Concerning IEEE Standards Documents

IEEE Standards documents are made available for use subject to important notices and legal disclaimers. These notices and disclaimers, or a reference to this page (<https://standards.ieee.org/ipr/disclaimers.html>), appear in all IEEE standards and may be found under the heading “Important Notices and Disclaimers Concerning IEEE Standards Documents.”

Notice and Disclaimer of Liability Concerning the Use of IEEE Standards Documents

IEEE Standards documents are developed within IEEE Societies and subcommittees of IEEE Standards Association (IEEE SA) Board of Governors. IEEE develops its standards through an accredited consensus development process, which brings together volunteers representing varied viewpoints and interests to achieve the final product. IEEE standards are documents developed by volunteers with scientific, academic, and industry-based expertise in technical working groups. Volunteers involved in technical working groups are not necessarily members of IEEE or IEEE SA and participate without compensation from IEEE. While IEEE administers the process and establishes rules to promote fairness in the consensus development process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the soundness of any judgments contained in its standards.

IEEE makes no warranties or representations concerning its standards, and expressly disclaims all warranties, express or implied, concerning all standards, including but not limited to the warranties of merchantability, fitness for a particular purpose and non-infringement. IEEE Standards documents do not guarantee safety, security, health, or environmental protection, or compliance with law, or guarantee against interference with or from other devices or networks. In addition, IEEE does not warrant or represent that the use of the material contained in its standards is free from patent infringement. IEEE Standards documents are supplied “AS IS” and “WITH ALL FAULTS.”

Use of an IEEE standard is wholly voluntary. The existence of an IEEE standard does not imply that there are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to the scope of the IEEE standard. Furthermore, the viewpoint expressed at the time a standard is approved and issued is subject to change brought about through developments in the state of the art and comments received from users of the standard.

In publishing and making its standards available, IEEE is not suggesting or rendering professional or other services for, or on behalf of, any person or entity, nor is IEEE undertaking to perform any duty owed by any other person or entity to another. Any person utilizing any IEEE Standards document should rely upon their own independent judgment in the exercise of reasonable care in any given circumstances or, as appropriate, seek the advice of a competent professional in determining the appropriateness of a given IEEE standard.

IN NO EVENT SHALL IEEE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO: THE NEED TO PROCURE SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE PUBLICATION, USE OF, OR RELIANCE UPON ANY STANDARD, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE AND REGARDLESS OF WHETHER SUCH DAMAGE WAS FORESEEABLE.

Translations

The IEEE consensus balloting process involves the review of documents in English only. In the event that an IEEE standard is translated, only the English language version published by IEEE is the approved IEEE standard.

Use by artificial intelligence systems

In no event shall material in any IEEE Standards documents be used for the purpose of creating, training, enhancing, developing, maintaining, or contributing to any artificial intelligence systems without the express, written consent of

IEEE SA in advance. “Artificial intelligence” refers to any software, application, or other system that uses artificial intelligence, machine learning, or similar technologies, to analyze, train, process, or generate content. Requests for consent can be submitted using the Contact Us form.

Official statements

A statement, written or oral, that is not processed in accordance with the IEEE SA Standards Board Operations Manual is not, and shall not be considered or inferred to be, the official position of IEEE or any of its committees and shall not be considered to be, or be relied upon as, a formal position of IEEE or IEEE SA. At lectures, symposia, seminars, or educational courses, an individual presenting information on IEEE standards shall make it clear that the presenter’s views should be considered the personal views of that individual rather than the formal position of IEEE, IEEE SA, the Standards Committee, or the Working Group. Statements made by volunteers may not represent the formal position of their employer(s) or affiliation(s). News releases about IEEE standards issued by entities other than IEEE SA should be considered the view of the entity issuing the release rather than the formal position of IEEE or IEEE SA.

Comments on standards

Comments for revision of IEEE Standards documents are welcome from any interested party, regardless of membership affiliation with IEEE or IEEE SA. However, IEEE does not provide interpretations, consulting information, or advice pertaining to IEEE Standards documents.

Suggestions for changes in documents should be in the form of a proposed change of text, together with appropriate supporting comments. Since IEEE standards represent a consensus of concerned interests, it is important that any responses to comments and questions also receive the concurrence of a balance of interests. For this reason, IEEE and the members of its Societies and subcommittees of the IEEE SA Board of Governors are not able to provide an instant response to comments or questions, except in those cases where the matter has previously been addressed. For the same reason, IEEE does not respond to interpretation requests. Any person who would like to participate in evaluating comments or revisions to an IEEE standard is welcome to join the relevant IEEE SA working group. You can indicate interest in a working group using the Interests tab in the Manage Profile & Interests area of the IEEE SA myProject system. An IEEE Account is needed to access the application.

Comments on standards should be submitted using the Contact Us form.

Laws and regulations

Users of IEEE Standards documents should consult all applicable laws and regulations. Compliance with the provisions of any IEEE Standards document does not constitute compliance to any applicable regulatory requirements. Implementers of the standard are responsible for observing or referring to the applicable regulatory requirements. IEEE does not, by the publication of its standards, intend to urge action that is not in compliance with applicable laws, and these documents may not be construed as doing so.

Data privacy

Users of IEEE Standards documents should evaluate the standards for considerations of data privacy and data ownership in the context of assessing and using the standards in compliance with applicable laws and regulations.

Copyrights

IEEE draft and approved standards are copyrighted by IEEE under U.S. and international copyright laws. They are made available by IEEE and are adopted for a wide variety of both public and private uses. These include both use by reference, in laws and regulations, and use in private self-regulation, standardization, and the promotion of engineering practices and methods. By making these documents available for use and adoption by public authorities and private users, neither IEEE nor its licensors waive any rights in copyright to the documents.

Photocopies

Subject to payment of the appropriate licensing fees, IEEE will grant users a limited, non-exclusive license to photocopy portions of any individual standard for company or organizational internal use or individual, non-commercial use only. To arrange for payment of licensing fees, please contact Copyright Clearance Center, Customer Service, 222 Rosewood Drive, Danvers, MA 01923 USA; +1 978 750 8400; <https://www.copyright.com/>. Permission to photocopy portions of any individual standard for educational classroom use can also be obtained through the Copyright Clearance Center.

Updating of IEEE Standards documents

Users of IEEE Standards documents should be aware that these documents may be superseded at any time by the issuance of new editions or may be amended from time to time through the issuance of amendments, corrigenda, or errata. An official IEEE document at any point in time consists of the current edition of the document together with any amendments, corrigenda, or errata then in effect.

Every IEEE standard is subjected to review at least every 10 years. When a document is more than 10 years old and has not undergone a revision process, it is reasonable to conclude that its contents, although still of some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that they have the latest edition of any IEEE standard.

In order to determine whether a given document is the current edition and whether it has been amended through the issuance of amendments, corrigenda, or errata, visit IEEE Xplore or contact IEEE. For more information about the IEEE SA or IEEE's standards development process, visit the IEEE SA Website.

Errata, if any, for all IEEE standards can be accessed on the IEEE SA Website. Search for standard number and year of approval to access the web page of the published standard. Errata links are located under the Additional Resources Details section. Errata are also available in IEEE Xplore. Users are encouraged to periodically check for errata.

Patents

IEEE standards are developed in compliance with the IEEE SA Patent Policy.

Attention is called to the possibility that implementation of this standard may require use of subject matter covered by patent rights. By publication of this standard, no position is taken by the IEEE with respect to the existence or validity of any patent rights in connection therewith. If a patent holder or patent applicant has filed a statement of assurance via an Accepted Letter of Assurance, then the statement is listed on the IEEE SA Website at <https://standards.ieee.org/about/sasb/patcom/patents.html>. Letters of Assurance may indicate whether the Submitter is willing or unwilling to grant licenses under patent rights without compensation or under reasonable rates, with reasonable terms and conditions that are demonstrably free of any unfair discrimination to applicants desiring to obtain such licenses.

Essential Patent Claims may exist for which a Letter of Assurance has not been received. The IEEE is not responsible for identifying Essential Patent Claims for which a license may be required, for conducting inquiries into the legal validity or scope of Patents Claims, or determining whether any licensing terms or conditions provided in connection with submission of a Letter of Assurance, if any, or in any licensing agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that determination of the validity of any patent rights, and the risk of infringement of such rights, is entirely their own responsibility. Further information may be obtained from the IEEE Standards Association.

IMPORTANT NOTICE

Technologies, application of technologies, and recommended procedures in various industries evolve over time. The IEEE standards development process allows participants to review developments in industries, technologies, and practices, and to determine what, if any, updates should be made to the IEEE standard. During this evolution, the tech-

nologies and recommendations in IEEE standards may be implemented in ways not foreseen during the standard's development. IEEE standards development activities consider research and information presented to the standards development group in developing any safety recommendations. Other information about safety practices, changes in technology or technology implementation, or impact by peripheral systems also may be pertinent to safety considerations during implementation of the standard. Implementers and users of IEEE Standards documents are responsible for determining and complying with all appropriate safety, security, environmental, health, data privacy, and interference protection practices and all applicable laws and regulations.

Participants

At the time this draft was completed, the P3109 Working Group had the following membership:

Kiran Gunnam, *Chair*
Leonard Tsai, *Vice Chair*
Jeffrey Sarnoff, *Secretary*

Malia Zaman, *IEEE Standards Board Liaison*
Tom Thompson, *past IEEE Standards Liaison*

Editors

Jeffrey Sarnoff (Editor in Chief), Andrew Fitzgibbon, Christoph M. Wintersteiger

Contributing Editors

Amos Omondi, Guy Lemieux

Contributors

Nima Badizadegan	Michel Hack	Michael Overton
Paul Balanca	Dandan Huang	Katherine Parry
David H. C. Chen	Laslo Hunhold	Nathalie Revol
Marco Cococcioni	Seokbum Ko	Jason Riedy
George Constantinides	Carlo Luschi	Ali Sazegari
Marius Cornea	David Lutz	Eric Schwarz
Mike Cowlshaw	Tue Ly	Olivier Sentieys
James Davenport	Albert Martin	Michael Siu
James Demmel	Mantas Mikaitis	Gil Tabak
Kenneth Dockser	Aaftab Munshi	Julio Villalba
Massimiliano Fasi	Santosh Nagarakatte	Kristopher Wong
Silviu-Ioan Filip	Aisha Naseer	Thomas Yeh
John Gustafson	Badreddine Nouné	Aleksandr Zakharchenko
	Stuart Oberman	

The following members of the individual Standards Association balloting group voted on this draft. Balloters may have voted for approval, disapproval, or abstention.

[To be supplied by IEEE.]

When the IEEE SA Standards Board approved this Standard on <Date Approved>, it had the following membership:

[To be supplied by IEEE.]

Trademarks and service marks: IEEE Std 754-2019™ is a trademark owned by The Institute of Electrical and Electronics Engineers, Incorporated.

Introduction

This introduction is not part of IEEE P3109/D1, Draft Standard for Arithmetic Formats for Machine Learning.

Machine learning systems increasingly use narrow floating-point formats to reduce energy use, conserve memory, and save computation time. Implementations have made differing choices concerning precision, signedness, special values, rounding, saturation, and scaling. These differences complicate data exchange and the consistent specification of arithmetic behavior across hardware and software. This standard provides a common framework intended to improve interoperability among machine learning systems and with dependent systems.

This standard defines a parameterized family of compact binary floating-point formats. Each format is specified by its bitwidth, precision, signedness, and domain. The standard also defines conversions, scalar operations, and operations on blocks of values that share a scale factor. Many numeric operations are specified through a common model that decodes operands to closed extended real values, performs a mathematical operation, and projects the result to a destination format using specified rounding and saturation modes. Conversions to and from selected IEEE Std 754-2019 binary formats connect this family with established floating-point environments.

The following objectives guided the development of this standard:

- a) Make the relevant properties of compact formats explicit through a small set of format-defining parameters.
- b) Provide consistent operation definitions for combinations of operand and result formats.
- c) Support a broad range of rounding modes, including directed rounding, round to odd, and stochastic rounding.
- d) Define methods for handling results outside a format's finite range, via saturation or overflow to infinity.
- e) Accommodate blocks of values that share a scale factor, including elementwise, reduction, and conversion operations.
- f) Provide a declaration framework for approximate implementations.

In machine learning environments, this standard is intended to provide a common basis for communication among numerical practitioners and designers of hardware, compilers, programming frameworks, and software libraries. Its format parameters, operation specializations, projection specifications, and conformance declarations make arithmetic behavior explicit without prescribing a particular hardware or software realization.

Contents

1	Overview	12
1.1	Scope	12
1.2	Word usage	12
2	Definitions and abbreviations	13
2.1	Definitions	13
2.2	Abbreviations	14
2.3	Mathematical notations	15
3	Floating-point formats	16
3.1	Formats	16
3.2	Naming	17
4	Operations	18
4.1	General	18
4.2	Projection specifications	18
4.3	Operation definitions	19
4.3.1	Operation definition schema	19
4.3.2	Pattern-matching declarations	19
4.4	Approximate implementations	21
4.5	Conforming implementations	22
4.6	Conformance declarations	24
4.7	Internal functions	25
4.7.1	General	25
4.7.2	Decoding	25
4.7.3	Projection	26
4.7.4	Rounding to precision	27
4.7.5	Saturation	29
4.7.6	Encoding	30
4.8	Decoding and encoding for external formats	31
4.8.1	Decoding	31
4.8.2	Encoding	31
4.9	Converting between formats	32
4.9.1	Conversion	32
4.10	Arithmetic operations	33
4.10.1	Absolute value, Negation	33
4.10.2	Copying the sign	34
4.10.3	Addition, Subtraction	35
4.10.4	Multiplication	36
4.10.5	Division	37
4.10.6	Fused Multiply–Add	38
4.10.7	Fused Add–Add	39
4.10.8	Square root, Reciprocal, Reciprocal square root	40
4.10.9	Logarithm, Exponentiation	41
4.10.10	Trigonometric functions	42
4.10.11	Hyperbolic functions	43
4.10.12	Trigonometric π -functions	44
4.10.13	Softplus	45
4.10.14	Hypotenuse	46
4.10.15	Inverse tangent of two variables	47

4.10.16	Inverse tangent of two variables (π -variant)	48
4.11	Extrema	49
4.11.1	Minimum and maximum, and number variants	49
4.11.2	Minimum and maximum magnitude, and number variants	50
4.11.3	Minimum and maximum finite variants	51
4.11.4	Clamping	52
4.12	Comparisons	53
4.12.1	Total order predicate	54
4.12.2	Comparison operator symbols	55
4.13	Predicates and classification	56
4.13.1	Classifier operation	57
4.14	Format-level operations	58
4.15	Projection specification operations	58
4.16	Next greater than and next less than	59
5	Block operations	61
5.1	General	61
5.2	Definitions	61
5.3	Stochastic rounding in block operations	61
5.4	Internal functions	62
5.4.1	Decoding	62
5.4.2	Projection	63
5.5	Conversion of blocks	64
5.5.1	Conversion from a block	64
5.5.2	Conversion to a block	65
5.5.3	Conversion to a block with scale factor computation	66
5.6	Block reduction operations	67
5.6.1	Sum and product	67
5.6.2	Dot product	68
5.7	Elementwise operations on blocks	69
5.8	Scaled operations via block operations	70
	Appendices	71
	Annex A (informative) Rationales and discussion	71
A.1	General	71
A.2	Not a number (NaN)	71
A.3	Zero	71
A.4	Infinities	72
A.5	Exponent bias	74
A.6	Subnormals	74
	Annex B (informative) Encoding	75
B.1	Value tables	76
	Annex C (informative) Value tables for K=2	78
	Annex D (informative) Examples of approximation declarations	79
D.1	Example: Approximate implementation of Exp	79
D.2	Example: Approximate implementation of BlockDotProduct	79
	Annex E (informative) Recommendations for reduction accuracy specifications	80
E.1	BlockReduceAdd	80
E.2	BlockReduceMultiply	80

E.3	BlockDotProduct	81
Annex F (informative) Operation groups		82
Annex G (informative) Bibliography		85

1 Overview

1.1 Scope

This standard defines a binary arithmetic and data format for machine learning-optimized domains. It also specifies the default handling of exceptions occurring in this arithmetic. This standard provides a consistent and flexible arithmetic framework optimized for Machine Learning Systems (MLSs) in hardware and/or software implementations to minimize the work required to make MLSs interoperable with each other as well as other dependent systems. This standard is aligned with the IEEE Std 754-2019 Standard for Floating-Point Arithmetic.

1.2 Word usage

The word *shall* indicates mandatory requirements strictly to be followed in order to conform to the standard and from which no deviation is permitted (*shall* equals *is required to*).^{1,2}

The word *should* indicates that among several possibilities one is recommended as particularly suitable, without mentioning or excluding others; or that a certain course of action is preferred but not necessarily required (*should* equals *is recommended that*).

The word *may* is used to indicate a course of action permissible within the limits of the standard (*may* equals *is permitted to*).

The word *can* is used for statements of possibility and capability, whether material, physical, or causal (*can* equals *is able to*).

¹The use of the word *must* is deprecated when stating mandatory requirements; *must* is used only to describe unavoidable situations.

²The use of *will* is deprecated when stating mandatory requirements; *will* is only used in statements of fact.

2 Definitions and abbreviations

2.1 Definitions

For the purposes of this document, the following terms and definitions apply. The *IEEE Standards Dictionary Online* should be consulted for terms not defined in this clause.³

bitwidth: the number of bits required to represent a floating-point value in a given format, denoted K .

block: a pair comprising a scale factor and a sequence of one or more floating-point values.

canonical form: of a nonzero finite floating-point datum X in format f , its representation as $\text{sgn}(X) \times S \times 2^{1-P} \times 2^E$ for minimal integer $E > -B$, and nonnegative integer $S < 2^P$, where $P = \text{PrecisionOf}(f)$ and $B = \text{ExponentBiasOf}(f)$. For zero, $S = 0$ and $E = 0$.

closed extended reals: set denoted $\mathbb{R}^\omega$, consisting of the real numbers augmented with positive infinity, negative infinity, and not-a-number: $\mathbb{R}^\omega := \mathbb{R} \cup \{-\infty, +\infty, \text{NaN}\}$.

code point: integer in the range 0 to $2^K - 1$ that encodes a floating-point datum occurring in a format of bitwidth K .

datum: see floating-point datum.

datum set: the subset of the closed extended reals representable in a given format f , denoted $\mathcal{D}_f$.

defined operation: operation whose signature and behavior are defined by this standard.

defined result: result of a defined operation for a given set of operands.

domain: format-defining parameter in $\{\text{Finite}, \text{Extended}\}$ that specifies whether the format's datum set includes infinities.

encoding: a format's unique mapping from its datum set to code points.

exponent: of a floating-point datum X expressed in canonical form as $\text{sgn}(X) \times S \times 2^{1-P} \times 2^E$, the integer E .

exponent bias: format-specific constant, denoted B , used in encoding and decoding the exponent field.

exponent field: the integer value of the biased exponent of a code point.

floating-point datum: a closed extended real value representable in a given format, an element of the datum set of the format.

floating-point format: binary floating-point representation parameterized by bitwidth, precision, signedness, and domain.

floating-point value: a code point representing a floating-point datum in a given format.

format-defining parameters: bitwidth (K), precision (P), signedness (Σ), and domain (Δ).

NaN (Not a Number): special non-numeric value used to represent undefined or unrepresentable results.

normal: finite floating-point datum whose significand S satisfies $2^{P-1} \leq S < 2^P$. A floating-point value is normal iff its datum is normal.

³IEEE Standards Dictionary Online is available at: <http://dictionary.ieee.org>. An IEEE account is required for access to the dictionary, and one can be created at no charge on the dictionary sign-in page.

operand: an argument to an operation.

operation: a set of operation specializations parameterized over operation parameters, e.g., format, projection specification, and constants such as block size.

operation specialization: mapping from a tuple of operands to one or more results.

precision: the bitwidth of the trailing significand field plus one, denoted P .

projection specification: pair (rounding mode, saturation mode) that determines how a general closed extended real is projected to a datum in a format's datum set.

rounded value: a closed extended real value, the result of rounding (via $\omega\text{RoundToPrecision}$, §4.7.4). If finite, it is of the form $S \times 2^E$ for integers S and E .

rounding mode: attribute of a projection specification that determines how a closed extended real value is rounded to a rounded value.

saturation mode: attribute of a projection specification that determines how rounded values outside the finite range of the target format are projected to a datum.

scale factor: multiplicative factor applied to all elements in a block.

significand: of a floating-point datum X expressed in canonical form as $\text{sgn}(X) \times S \times 2^{1-P} \times 2^E$, the integer S .

signed format: format that represents signed numbers and has signedness parameter Signed.

signedness: format-defining parameter in $\{\text{Signed}, \text{Unsigned}\}$ that specifies whether the format includes negative datums.

special value: floating-point values encoding zero, $\pm\infty$, and NaN.

subnormal: finite floating-point datum whose significand S satisfies $0 < S < 2^{P-1}$. A floating-point value is subnormal iff its datum is subnormal.

trailing significand field: of a code point x in format f , the value $x \bmod 2^{P-1}$ where P is the precision of the format f .

unsigned format: format that represents non-negative numbers and has signedness parameter Unsigned.

value set: the set of floating-point values for a format.

2.2 Abbreviations

FAA	fused add-add
FMA	fused multiply-add
MLS	machine learning system
P3109	IEEE Working Group P3109 (Standard for Arithmetic Formats for Machine Learning)
IEEE-754	IEEE Std 754-2019™

2.3 Mathematical notations

$\text{IsOdd}(I)$ is true if integer I is odd, false otherwise.

$\text{IsEven}(I)$ is true if integer I is even, false otherwise.

The number of elements in a set S is denoted $\#S$.

The signum function $\text{sgn}(X)$ is defined as -1 for $X < 0$, 0 for $X = 0$, and $+1$ for $X > 0$.

Division of nonnegative integers is denoted by $x \div y$, modulo by $x \bmod y$.

The notation $x_{i..j}$ denotes the sequence $[x_i, x_{i+1}, \dots, x_j]$.

Comments are right-justified and parenthesized.

(A comment)

3 Floating-point formats

3.1 Formats

Define the set of *closed extended reals* to be the reals augmented with positive and negative infinity and NaN:

$$\mathbb{R}^\omega := \mathbb{R} \cup \{-\infty, +\infty, \text{NaN}\}$$

NOTE 1—This set contains a single NaN value (see Annex A.2). There is no negative zero in this set (see Annex A.3).

A *floating-point format* f comprises a *datum set* $\mathcal{D}_f$ and an *encoding*. The datum set is a subset of the closed extended reals. The *encoding* is a unique bijective mapping from $\mathcal{D}_f$ to integer *code points* $0 \dots 2^K - 1$, where K is the bitwidth of the format.

A floating-point format has four *format-defining* parameters:

- Bitwidth K , an integer greater than two;⁴
- Precision P , an integer greater than zero. P shall be strictly less than K ($0 < P < K$) for signed formats, and less than or equal to K ($0 < P \leq K$) for unsigned formats;
- Signedness Σ , in $\{\text{Signed}, \text{Unsigned}\}$;
- Domain Δ , in $\{\text{Finite}, \text{Extended}\}$.⁵

The *exponent bias* is derived from the format-defining parameters.⁶ For signed formats, the exponent bias shall be $B = 2^{K-P-1}$. For unsigned formats, the exponent bias shall be $B = 2^{K-P}$.

A *floating-point datum* in a format f is an element of $\mathcal{D}_f$, i.e., a closed extended real that encodes to a code point in f .

A *floating-point value* is a code point representing a floating-point datum in a given format. The code points for infinities are denoted Inf and $-\text{Inf}$.

NOTE 2—While a floating-point value is synonymous with a code point, the term is used in contexts where a format is associated with the value. For example, in an operation definition, a value might be declared “ x : floating-point value, format f ”. In such contexts, the interpretation of a real constant, e.g., 2.0, as a floating-point value is to be read as $\omega\text{Encode}_f(2.0)$. In cases where the format is ambiguous, a real constant is subscripted with its associated format, e.g., $X_f = \omega\text{Encode}_f(X)$. For example, $2.0_{\text{Binary8p4se}} = \omega\text{Encode}_{\text{Binary8p4se}}(2.0)$ represents the code point 0x48. Similarly, $\text{Inf}_{\text{Binary8p4se}} = \infty_{\text{Binary8p4se}}$ represents the code point 0x7F.

The word *finite* applied to a datum signifies that the datum is not $\pm\infty$ or NaN. Applied to a value, it signifies that the value encodes a finite datum.

A finite floating-point datum is a real number whose absolute value has the form

$$S \times 2^{1-P} \times 2^E,$$

where the *exponent* E is an integer such that $E > -B$, and S , the *significand*, is an integer $0 \leq S < 2^P$. This decomposition is made canonical for nonzero values by choosing the smallest $E > -B$. For $S = 0$, E is chosen to be 0. The datum is *normal* if $2^{P-1} \leq S < 2^P$. The datum is *subnormal*⁷ if $0 < S < 2^{P-1}$. The datum zero is neither normal nor subnormal.

The *trailing significand* is the integer $S \bmod 2^{P-1}$.

NOTE 3—The encoding of formats is described in §4.7.6; properties of encodings are described in Annex B.

⁴See Annex C for rationale for $K > 2$.

⁵See Annex A.4 for rationale for parameterization of domain.

⁶See Annex A.5 for rationale for this choice of exponent bias.

⁷See Annex A.6 for rationale for inclusion of subnormals.

3.2 Naming

Formats defined in this document shall be named $\text{Binary}\{K, P, \Sigma, \Delta\}$.

A shortened notation is also used to refer to specific formats: $\text{Binary}\langle\kappa\rangle\text{p}\langle\psi\rangle\langle\sigma\rangle\langle\delta\rangle$ where the placeholders $\langle\kappa\rangle$ and $\langle\psi\rangle$ are decimal representations of the bitwidth K and precision P , respectively; signedness $\langle\sigma\rangle \in \{\text{s}, \text{u}\}$, for signed and unsigned; and domain $\langle\delta\rangle \in \{\text{e}, \text{f}\}$ for extended and finite.

Examples:

- The format “Binary12p7se” is a 12-bit signed, extended format with precision 7.
- The format “Binary8p1uf” is an 8-bit unsigned, finite format with precision 1.

NOTE—The term “binary” indicates that the finite datums are integer multiples of (positive or negative) powers of two.

4 Operations

4.1 General

An *operation* is a parameterized mapping from a tuple of operands to one or more *results*. An operation is denoted by an identifier, followed by a list of *operation parameter* names, either as subscripts or between angle brackets.

An *operation specialization* supplies values for the parameters of an operation. An operation specialization is denoted by an identifier, followed by a list of operation parameter values, either as subscripts or between angle brackets.

Example:

$\text{Exp}\langle f_x, f_r, \rho \rangle$ is an operation, and $\text{Exp}\langle \text{Binary8p4se}, \text{Binary8p3ue}, (\text{NearestTiesToEven}, \text{SatNone}) \rangle$ is a specialization of this operation.

In some cases, an *auxiliary operation* is defined, which operates on the closed extended reals. Such operations are named with the prefix ω . These operations handle non-finite values explicitly and immediately; all arithmetic and other mathematical operations are over (finite) real values.

NOTE 1—Specifications operate on code points using integer arithmetic (e.g., divide and modulo) operations. These operations may be performed in any equivalent manner, for example bit shifting and masking.

NOTE 2—Operations on floating-point values are defined via conversion to the closed extended reals, on which the mathematical operation is performed, before projection into the floating-point datum set via rounding and saturation. The definitions in this document are specifications of behavior.

NOTE 3—The operation definitions herein describe no side effects, such as the setting of flags, or the triggering of interrupts, and do not return values other than the defined result. Generally, NaN is returned when operands are out of domain (e.g., $\text{Log}(-1.0)$).

NOTE 4—Many properties of the formats and operations are formally verified [12, 2]. Auxiliary operations are automatically extracted from a formal specification [12].

4.2 Projection specifications

A *projection specification* is a pair (rounding mode, saturation mode).

Rounding modes, as specified in §4.7.4, describe the mapping of closed extended real values to *rounded values* as follows:

NearestTiesToEven	Round to nearest, ties to even
NearestTiesToAway	Round to nearest, ties away from zero
TowardPositive	Round toward positive
TowardNegative	Round toward negative
TowardZero	Round toward zero
ToOdd	Round to odd
Stochastic[A, B, C]	Stochastic rounding, with variants A, B, and C as defined in §4.7.4

Saturation modes, as specified in §4.7.5, describe the mapping of rounded values to datums as follows:

SatFinite	All rounded values are clamped to the representable finite range.
SatPropagate	Infinite rounded values are preserved if representable. All other rounded values are clamped to the representable range.
SatNone	Out-of-range rounded values are replaced with the extremal finite datum, positive or negative infinity, or NaN, as governed by the rounding direction and the signedness of the target format.

4.3 Operation definitions

4.3.1 Operation definition schema

Operations are defined according to the following schema.

Signature

$\text{Operation}_{p_1, \dots}(x_1, \dots) \rightarrow r$

Parameters

p_1 : description of parameter 1

...

Operands

x_1 : description of operand 1

...

Result

r : description of result

Behavior

An ordered sequence of *pattern-matching declarations* (see §4.3.2)

Details

An optional section listing additional aspects of the operation's behavior.

4.3.2 Pattern-matching declarations

A pattern-matching declaration is of the form

$\text{Operation}(x_1, \dots) \rightarrow \dots$

describing the result of the operation given operands $x_1, \dots$

In a sequence of pattern-matching declarations, the first matching pattern in the order presented in this document defines the behavior for given operands.

The following notations facilitate concise specifications.

4.3.2.1 Exact pattern

Only the provided operands match.

Example:

$\text{Operation}(\text{NaN}, 0) \rightarrow \text{NaN}$

4.3.2.2 Set inclusion

Match for all x values in the given set.

Example:

$\text{Operation}(x \in \{-\text{Inf}, \text{Inf}\}, y) \rightarrow x$

4.3.2.3 Wildcard match

The $*$ symbol matches any value.

Example:

$\text{Operation}(*, 0) \rightarrow 0$

4.3.2.4 Explicit parameters introduced in result expression

A reference to an operation specialization in the result expression uses explicit parameters.

Example:

$\text{Operation}(x, y) \rightarrow \text{Operation}_{f_x, f_y, f_r}(x, y)$

Example:

The definition of the operation Log, taken from §4.10.9:

Signature

$\text{Log}_{f_x, f_r, \rho}(x) \rightarrow r$

Parameters

f_x : format of operand x

f_r : format of result r

ρ : projection specification

Operands

x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$\omega\text{Log}(\text{NaN}) \rightarrow \text{NaN}$

$\omega\text{Log}(-\infty) \rightarrow \text{NaN}$

$\omega\text{Log}(+\infty) \rightarrow +\infty$

$\omega\text{Log}(X) \text{ if } X < 0 \rightarrow \text{NaN}$

$\omega\text{Log}(0) \rightarrow -\infty$

$\omega\text{Log}(X) \rightarrow \log_e X$

$\text{Log}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Log}(\omega\text{Decode}_{f_x}(x)))$

4.4 Approximate implementations

An operation is a *numeric operation* if one or more of its results is a floating-point value.

An operation's *defined results* are the result values specified in the definitions in this document.

For numeric operations, a system that provides approximate implementations shall declare them as κ -approximate operations as follows.⁸ A κ -approximate implementation shall compute values whose maximum difference from the defined results does not exceed κ value steps, as described in this section.

A numeric operation a , for a given set of parameters, has a defined result $\hat{a}(x)$ for operands x . A κ -approximate implementation produces a floating-point value $\tilde{a}(x)$, which for some operands x has $\tilde{a}(x) \neq \hat{a}(x)$.

Where an approximate implementation does not “match on NaNs”, that is where $\text{IsNaN}(\hat{a}(x)) \neq \text{IsNaN}(\tilde{a}(x))$ for some input x , then it shall declare $\kappa = \text{NaN}$.

Define

$$\text{MatchOnInfinity}(x, y) = \begin{cases} \text{True} & \text{if } \text{IsNaN}(x) \text{ and } \text{IsNaN}(y) \\ \text{True} & \text{if } \text{IsFinite}(x) \text{ and } \text{IsFinite}(y) \\ \text{True} & \text{if } \text{IsInfinite}(x) \text{ and } \text{IsInfinite}(y) \text{ and } \text{IsSignMinus}(x) = \text{IsSignMinus}(y) \\ \text{False} & \text{otherwise} \end{cases}$$

Where an approximate implementation matches on NaNs but does not match on infinities, that is where $\text{MatchOnInfinity}(\hat{a}(x), \tilde{a}(x))$ is false for some input x , then it shall declare $\kappa = \infty$.

For operations which match on NaNs and infinities, the value of κ is defined as follows.

Let the set of operands producing finite results be I , so that for all $x \in I$ we have $\hat{a}(x) \in V$, where V is the result format's finite value set. For all $x \in I$, $\tilde{a}(x)$ shall be in V .

The value of κ is the maximum over all operands $x \in I$ of the number of values in V between $\tilde{a}(x)$ and $\hat{a}(x)$ inclusive of the former, exclusive of the latter. Formally,⁹

$$\kappa = \max_{x \in I} \# \left(\left((\hat{a}(x), \tilde{a}(x)] \cup [\tilde{a}(x), \hat{a}(x)) \right) \cap V \right).$$

The value of κ is, in general, specific to each operation specialization. For example a system might supply implementations of $\text{Add}_{f_x, f_y, f_r, \rho}$ where κ depends on f_r .

For each κ -approximate operation specialization, κ shall be specified. Such a specification may be over I or over a covering union of disjoint subsets of I , written

$$\kappa \in \{I_1 : \kappa_{I_1}, \dots, I_n : \kappa_{I_n}\} \quad \text{where} \quad I = \bigcup_i I_i \quad \text{and} \quad \forall i \neq j : I_i \cap I_j = \{\}.$$

This can be attested by any proof method, including direct computation.

Operations returning multiple floating-point values shall declare κ as follows, where $\kappa[m]$ is κ for the operation which returns the m^{th} return value, that is $\tilde{a}(x)[m]$. If any $\kappa[m]$ is NaN, $\kappa = \text{NaN}$ shall be declared. Otherwise, if any $\kappa[m]$ is infinite, $\kappa = \infty$ shall be declared. If all $\kappa[m]$ are finite, $\kappa = \max_m \kappa[m]$ shall be declared.

Approximate implementations shall be named differently from the name of the operation that is approximated.

Additional accuracy declarations may be supplied.¹⁰

⁸See Annex D for worked examples of approximation declarations.

⁹The intervals $(\ell, u]$ and $[\ell, u)$ where $u \leq \ell$ are empty sets.

¹⁰See Annex E for examples.

4.5 Conforming implementations

An implementation shall provide the following operation specializations:

Convert< f, f_r, ρ >	where	$\{f, f_r\} \subset \mathcal{F}_4 \cup \mathcal{F}_8 \cup \mathcal{F}_X$	
Negate< f, f, ρ >	where	$f \in \mathcal{F}_4 \cup \mathcal{F}_8$	
Abs< f, f, ρ >	where	$f \in \mathcal{F}_4 \cup \mathcal{F}_8$	
Recip< f, f_r, ρ >	where	$\{f, f_r\} \subset \mathcal{F}_4 \cup \mathcal{F}_8 \cup \mathcal{F}_X$	
Add< f_1, f_2, f_r, ρ >	where	$\{f_1, f_2\} \subset \mathcal{F}_4 \cup \mathcal{F}_8$ and $f_r \in \mathcal{F}_8 \cup \mathcal{F}_X$	
Subtract< f_1, f_2, f_r, ρ >	where	$\{f_1, f_2\} \subset \mathcal{F}_4 \cup \mathcal{F}_8$ and $f_r \in \mathcal{F}_8 \cup \mathcal{F}_X$	
Multiply< f_1, f_2, f_r, ρ >	where	$\{f_1, f_2\} \subset \mathcal{F}_4 \cup \mathcal{F}_8$ and $f_r \in \mathcal{F}_8 \cup \mathcal{F}_X$	
FMA< f_1, f_2, f_r, f_r, ρ >	where	$\{f_1, f_2\} \subset \mathcal{F}_4 \cup \mathcal{F}_8$ and $f_r \in \mathcal{F}_X$	
FAA< f_1, f_2, f_r, f_r, ρ >	where	$\{f_1, f_2\} \subset \mathcal{F}_4 \cup \mathcal{F}_8$ and $f_r \in \mathcal{F}_X$	
MinmaxOp< f, f, f, ρ >	where	$f \in \mathcal{F}_4 \cup \mathcal{F}_8$	
	and	$\text{MinmaxOp} \in \left\{ \begin{array}{ll} \text{Minimum,} & \text{Maximum,} \\ \text{MinimumNumber,} & \text{MaximumNumber,} \\ \text{MinimumMagnitude,} & \text{MaximumMagnitude,} \\ \text{MinimumMagnitudeNumber,} & \text{MaximumMagnitudeNumber,} \\ \text{MinimumFinite,} & \text{MaximumFinite} \end{array} \right\}$	

where the two format sets $\mathcal{F}_4$ and $\mathcal{F}_8$ are defined as

$$\begin{aligned} \mathcal{F}_8 &= \{\text{Binary8p4se}, \text{Binary8p3se}\} \\ \mathcal{F}_4 &= \{\text{Binary4p2sf}\} \end{aligned}$$

and the external format¹¹ set $\mathcal{F}_X$ is defined as

$$\{\} \neq \mathcal{F}_X \subseteq \{\text{binary32}, \text{binary16}, \text{BFloat16}\}$$

and where $\rho = (\text{NearestTiesToEven}, \text{SatNone})$.

For $\text{CompareOp} \in \{\text{CompareLess}, \text{CompareLessEqual}, \text{CompareEqual}, \text{CompareGreater}, \text{CompareGreaterEqual}\}$ the following operation specializations shall be provided:

$\text{CompareOp}<f, f>$ **where** $f \in \mathcal{F}_4 \cup \mathcal{F}_8$.

For each format in $\mathcal{F}_4 \cup \mathcal{F}_8$, the following operation specializations shall be provided:

IsZero, IsOne, IsNaN, IsInfinite, IsFinite, IsSignMinus, IsNormal, IsSubnormal, NextGreaterThan, NextLessThan.

For each format in $\mathcal{F}_4 \cup \mathcal{F}_8 \cup \mathcal{F}_X$, the following format-level operation specializations shall be provided:

BitwidthOf, PrecisionOf, SignednessOf, DomainOf,
ExponentBitwidthOf, TrailingSignificandBitwidthOf, ExponentBiasOf,
MaxFiniteOf, MinFiniteOf, MinPositiveOf, MaxSubnormalOf, MinNormalOf.

¹¹ See §4.14 about external formats.

In addition, scaled operations, as defined in §5.8, shall be provided as follows, with $\mathcal{F}_s = \{\text{Binary8p1uf}\}$:

ScaledAdd	$\langle (f_s, f_1), (f_s, f_2), f_r, \rho \rangle$	where	$f_s \in \mathcal{F}_s$
		and	$\{f_1, f_2\} \subset \mathcal{F}_4 \cup \mathcal{F}_8$
		and	$f_r \in \mathcal{F}_8 \cup \mathcal{F}_X$
ScaledSubtract	$\langle (f_s, f_1), (f_s, f_2), f_r, \rho \rangle$	where	$f_s \in \mathcal{F}_s$
		and	$\{f_1, f_2\} \subset \mathcal{F}_4 \cup \mathcal{F}_8$
		and	$f_r \in \mathcal{F}_8 \cup \mathcal{F}_X$
ScaledMultiply	$\langle (f_s, f_1), (f_s, f_2), f_r, \rho \rangle$	where	$f_s \in \mathcal{F}_s$
		and	$\{f_1, f_2\} \subset \mathcal{F}_4 \cup \mathcal{F}_8$
		and	$f_r \in \mathcal{F}_8 \cup \mathcal{F}_X$

Note that the choice of subset for $\mathcal{F}_X$ is implementation-defined.

4.6 Conformance declarations

Note that an implementation provides a subset of operation specializations for a subset of operations defined in this standard.

Where an operation specialization is supplied, the implementation shall compute the same result as does the defined operation specialization for all possible operand values. This should be attested by any appropriate proof method, including direct computation.

Where an approximate implementation of an operation specialization is supplied, the implementation shall declare κ as defined in §4.4. Such an implementation should use an implementation-specific identifier which indicates that the implementation is approximate.

An implementation should supply a means whereby a user can query the presence of an implementation of a given operation specialization defined by a string representation.

Example:

An implementation that supports an exponentiation operation which operates on 8-bit, signed, extended floating-point values with a precision of 3 or 4, with result precision always 4, and saturation to finite or no saturation. These operation specializations could be declared as follows:

`exp_8.3.to.8.4.finite: Exp<Binary8p3se, Binary8p4se, (NearestTiesToEven, SatFinite)>`

`exp_8.4.to.8.4.finite: Exp<Binary8p4se, Binary8p4se, (NearestTiesToEven, SatFinite)>`

`exp_8.3.to.8.4.inf: Exp<Binary8p3se, Binary8p4se, (NearestTiesToEven, SatNone)>`

`exp_8.4.to.8.4.inf: Exp<Binary8p4se, Binary8p4se, (NearestTiesToEven, SatNone)>`

4.7 Internal functions

4.7.1 General

The following subclauses define auxiliary operations which are referenced in the definitions of operations, but which themselves are not required of a conforming implementation.

4.7.2 Decoding

Signature

$$\omega\text{Decode}_f(x) \rightarrow X$$

Parameters

f : format of x

Operands

x : floating-point value, in format f

Result

X : floating-point datum, a closed extended real value in $\mathcal{D}_f$

Behavior

$\omega\text{Decode}(x) \rightarrow \omega\text{DecodeExternal}_f(x)$ **if** $f \in \{\text{binary64}, \text{binary32}, \text{binary16}, \text{BFloat16}\}$ ¹²
 $\omega\text{Decode}(x) \rightarrow \omega\text{DecodeAux}_f(\text{SignednessOf}(f), \text{DomainOf}(f), x)$

$\omega\text{DecodeAux}(\text{Signed}, *, 2^{K-1}) \rightarrow \text{NaN}$

$\omega\text{DecodeAux}(\text{Unsigned}, *, 2^K - 1) \rightarrow \text{NaN}$

$\omega\text{DecodeAux}(\text{Signed}, \text{Extended}, 2^{K-1} - 1) \rightarrow +\infty$

$\omega\text{DecodeAux}(\text{Signed}, \text{Extended}, 2^K - 1) \rightarrow -\infty$

$\omega\text{DecodeAux}(\text{Unsigned}, \text{Extended}, 2^K - 2) \rightarrow +\infty$

$\omega\text{DecodeAux}(\text{Signed}, \Delta, 2^{K-1} < x < 2^K) \rightarrow -\omega\text{DecodeAux}_f(\text{Signed}, \Delta, x - 2^{K-1})$

$\omega\text{DecodeAux}(*, *, x) \rightarrow X$

where

$$T = x \bmod 2^{P-1}$$

(Trailing significand)

$$E_{\text{biased}} = x \div 2^{P-1}$$

(Biased exponent)

$$X = \begin{cases} (0 + T \times 2^{1-P}) \times 2^{1-B} & \text{if } E_{\text{biased}} = 0 \\ (1 + T \times 2^{1-P}) \times 2^{E_{\text{biased}} - B} & \text{otherwise} \end{cases}$$

(Subnormal and zero)

(Normal)

and

$P = \text{PrecisionOf}(f)$

$B = \text{ExponentBiasOf}(f)$

where

$K = \text{BitwidthOf}(f)$

¹²See §4.14 about external formats.

4.7.3 Projection

Project closed extended real value to format f , applying specified rounding and saturation.

Signature

$$\omega\text{Project}_{f,\rho}(X) \rightarrow x$$

Parameters

f : target format

ρ : projection specification

Operands

X : closed extended real value

Result

x : floating-point value in format f

Behavior

$$\omega\text{Project}(\text{NaN}) \rightarrow \text{NaN}$$

$$\omega\text{Project}(X) \rightarrow x$$

where

$$R = \omega\text{RoundToPrecision}_{\text{PrecisionOf}(f), \text{ExponentBiasOf}(f), \text{RoundOf}(\rho)}(X)$$

$$S = \omega\text{Saturate}_{M^{\text{lo}}, M^{\text{hi}}}(\text{SatOf}(\rho), \text{RoundOf}(\rho), R, \Sigma, \Delta)$$

$$x = \omega\text{Encode}_f(S)$$

and

$$\Sigma = \text{SignednessOf}(f)$$

$$\Delta = \text{DomainOf}(f)$$

$$M^{\text{lo}} = \omega\text{Decode}_f(\text{MinFiniteOf}(f))$$

$$M^{\text{hi}} = \omega\text{Decode}_f(\text{MaxFiniteOf}(f))$$

NOTE 1—Under NearestTiesToEven rounding, $\omega\text{Project}$ has the property, shared with IEEE-754, that values between M^{hi} and the number half a unit in the last place above M^{hi} will project to M^{hi} because $\omega\text{RoundToPrecision}$ precedes saturation.

NOTE 2—Under NearestTiesToEven rounding, subnormals are handled following IEEE-754, for example, values strictly between the smallest subnormal and its half are rounded to the smallest subnormal, while values of magnitude below or equal to half that of the smallest subnormal are rounded to zero.

4.7.4 Rounding to precision

Convert a closed extended real value to a closed extended real value expressible (where finite) as an integer multiple of a power of two.

NOTE— Where X is nonzero and finite, the returned value with a given precision P and exponent bias B , is of the form $\text{sgn}(X) \times S \times 2^{1-P} \times 2^E$, where $S \in \mathbb{Z}$, $0 \leq S < 2^P$, and $E \in \mathbb{Z}$ with $-B < E$.

Signature

$\omega\text{RoundToPrecision}_{P,B,\mu}(X) \rightarrow Z$

Parameters

P : precision
 B : exponent bias
 μ : rounding mode

Operands

X : closed extended real value

Result

Z : closed extended real value

Behavior

$\omega\text{RoundToPrecision}(X \in \{0, -\infty, +\infty, \text{NaN}\}) \rightarrow X$

$\omega\text{RoundToPrecision}(X) \rightarrow Z$

where

$Q = \max(\lfloor \log_2(|X|) \rfloor, 1 - B) - P + 1$ (Subnormals handled by $\max(\cdot, 1 - B)$)

$\tilde{S} = |X| \times 2^{-Q}$ (Real-valued significand, to be rounded to integer)

$S = \begin{cases} \lfloor \tilde{S} \rfloor + 1 & \text{if RoundAway}(\mu) \\ \lfloor \tilde{S} \rfloor & \text{otherwise} \end{cases}$

$Z = \text{sgn}(X) \times S \times 2^Q$

and, for $\nu = \tilde{S} - \lfloor \tilde{S} \rfloor$:

$\text{RoundAway}(\text{TowardZero}) = \text{False}$

$\text{RoundAway}(\text{TowardPositive}) = \nu > 0 \text{ and } X > 0$

$\text{RoundAway}(\text{TowardNegative}) = \nu > 0 \text{ and } X < 0$

$\text{RoundAway}(\text{NearestTiesToAway}) = \nu \geq 0.5$

$\text{RoundAway}(\text{NearestTiesToEven}) = \nu > 0.5 \text{ or } (\nu = 0.5 \text{ and not CodelsEven})$

$\text{RoundAway}(\text{ToOdd}) = \nu > 0 \text{ and CodelsEven}$

$\text{RoundAway}(\text{StochasticA}_{N,R}) = \lfloor \nu \times 2^N \rfloor + R \geq 2^N$

$\text{RoundAway}(\text{StochasticB}_{N,R}) = \lfloor \nu \times 2^{N+1} \rfloor + (2 \times R + 1) \geq 2^{N+1}$

$\text{RoundAway}(\text{StochasticC}_{N,R}) = \text{RNITE}(\nu \times 2^N) + R \geq 2^N$

and

$\text{CodelsEven} = \begin{cases} \text{IsEven}(\lfloor \tilde{S} \rfloor) & \text{if } P > 1 \\ (\lfloor \tilde{S} \rfloor = 0) \text{ or } \text{IsEven}(Q + B) & \text{if } P = 1 \end{cases}$

$\text{RNITE}(X) = \begin{cases} \lfloor X \rfloor & \text{if } (X < \lfloor X \rfloor + 0.5) \text{ or } (X = \lfloor X \rfloor + 0.5 \text{ and IsEven}(\lfloor X \rfloor)) \\ \lfloor X \rfloor + 1 & \text{otherwise} \end{cases}$

Details

In stochastic rounding, random bits R are supplied as an unsigned integer in the range $0 \leq R < 2^N$, thus N is the number of random bits.

NOTE 1—The quality of the random bits is not specified in this document.

NOTE 2—The variants StochasticA, StochasticB, StochasticC offer a balance between accuracy and complexity [3].

NOTE 3—The notation $\text{StochasticA}_{N,*}$ is used to indicate that N random bits are supplied but not further specified.

4.7.5 Saturation

Saturate closed extended real to $\pm\infty$, or to maximum/minimum finite value.

Signature

$$\omega\text{Saturate}_{M^{\text{lo}}, M^{\text{hi}}}(\text{Sat}, \text{Round}, X, \Sigma, \Delta) \rightarrow Z$$

Parameters

M^{lo} : minimum finite value

M^{hi} : maximum finite value

Operands

Sat : saturation mode

Round : rounding mode

X : closed extended real value

Σ : signedness in {Signed, Unsigned}

Δ : domain in {Finite, Extended}

Result

Z : closed extended real value

Behavior

$\omega\text{Saturate}(*, *, \text{NaN}, *, *) \rightarrow \text{NaN}$

$\omega\text{Saturate}(*, *, X, *, *) \text{ if } M^{\text{lo}} \leq X \text{ and } X \leq M^{\text{hi}} \rightarrow X$

$\omega\text{Saturate}(\text{SatFinite}, *, +\infty, *, *) \rightarrow M^{\text{hi}}$

$\omega\text{Saturate}(\text{SatFinite}, *, -\infty, *, *) \rightarrow M^{\text{lo}}$

$\omega\text{Saturate}(\text{SatFinite}, *, X, *, *) \text{ if } X < M^{\text{lo}} \rightarrow M^{\text{lo}}$

$\omega\text{Saturate}(\text{SatFinite}, *, X, *, *) \text{ if } X > M^{\text{hi}} \rightarrow M^{\text{hi}}$

$\omega\text{Saturate}(\text{SatPropagate}, *, +\infty, *, \text{Extended}) \rightarrow +\infty$

$\omega\text{Saturate}(\text{SatPropagate}, *, +\infty, *, *) \rightarrow M^{\text{hi}}$

$\omega\text{Saturate}(\text{SatPropagate}, *, -\infty, \text{Signed}, \text{Extended}) \rightarrow -\infty$

$\omega\text{Saturate}(\text{SatPropagate}, *, -\infty, *, *) \rightarrow M^{\text{lo}}$

$\omega\text{Saturate}(\text{SatPropagate}, *, X, *, *) \text{ if } X < M^{\text{lo}} \rightarrow M^{\text{lo}}$

$\omega\text{Saturate}(\text{SatPropagate}, *, X, *, *) \text{ if } X > M^{\text{hi}} \rightarrow M^{\text{hi}}$

$\omega\text{Saturate}(\text{SatNone}, \text{TowardZero or TowardNegative}, X, *, *) \text{ if } X > M^{\text{hi}} \text{ and } X \neq +\infty \rightarrow M^{\text{hi}}$

$\omega\text{Saturate}(\text{SatNone}, \text{TowardZero or TowardPositive}, X, *, *) \text{ if } X < M^{\text{lo}} \text{ and } X \neq -\infty \rightarrow M^{\text{lo}}$

$\omega\text{Saturate}(\text{SatNone}, *, +\infty, *, \text{Extended}) \rightarrow +\infty$

$\omega\text{Saturate}(\text{SatNone}, *, -\infty, \text{Signed}, \text{Extended}) \rightarrow -\infty$

$\omega\text{Saturate}(\text{SatNone}, *, -\infty, \text{Unsigned}, \text{Extended}) \rightarrow \text{NaN}$

$\omega\text{Saturate}(\text{SatNone}, *, X, \text{Signed}, \text{Extended}) \text{ if } X < M^{\text{lo}} \rightarrow -\infty$

$\omega\text{Saturate}(\text{SatNone}, *, X, \text{Unsigned}, \text{Extended}) \text{ if } X < M^{\text{lo}} \rightarrow \text{NaN}$

$\omega\text{Saturate}(\text{SatNone}, *, X, *, \text{Extended}) \text{ if } X > M^{\text{hi}} \rightarrow +\infty$

$\omega\text{Saturate}(\text{SatNone}, *, *, *, \text{Finite}) \rightarrow \text{NaN}$

NOTE—Whilst rounding precedes saturation in $\omega\text{Project}$, the rounding mode is supplied to $\omega\text{Saturate}$ in order to resolve cases where rounding direction requires a finite result. Saturation does not round any value.

4.7.6 Encoding

Encode a closed extended real value to a code point in format f .

Signature

$\omega\text{Encode}_f(X) \rightarrow r$

Parameters

f : target format

Operands

X : floating-point datum, a closed extended real value in the datum set of format f

Result

r : floating-point value, in format f

Behavior

$\omega\text{Encode}(X) \rightarrow \omega\text{EncodeExternal}_f(X)$ if $f \in \{\text{binary64}, \text{binary32}, \text{binary16}, \text{BFloat16}\}$

$$\omega\text{Encode}(\text{NaN}) \rightarrow \begin{cases} 2^{K-1} & \text{if } \Sigma = \text{Signed} \\ 2^K - 1 & \text{if } \Sigma = \text{Unsigned} \end{cases}$$

$$\omega\text{Encode}(+\infty) \rightarrow \begin{cases} 2^{K-1} - 1 & \text{if } \Sigma = \text{Signed} \\ 2^K - 2 & \text{if } \Sigma = \text{Unsigned} \end{cases}$$

$\omega\text{Encode}(X < 0) \rightarrow \omega\text{Encode}_f(-X) + 2^{K-1}$

$\omega\text{Encode}(0) \rightarrow 0$

$\omega\text{Encode}(X > 0) \rightarrow r$

where

$$r = \begin{cases} T & \text{if } S < 2^{P-1} \\ T + (E + B) \times 2^{P-1} & \text{otherwise} \end{cases} \quad (\text{Subnormals})$$

and

$E = \max(\lfloor \log_2(X) \rfloor, 1 - B)$

$S = X \times 2^{-E} \times 2^{P-1}$ (S is the significand)

$T = S \bmod 2^{P-1}$

and

$K = \text{BitwidthOf}(f)$

$P = \text{PrecisionOf}(f)$

$B = \text{ExponentBiasOf}(f)$

$\Sigma = \text{SignednessOf}(f)$

NOTE— ωEncode is applied only to a value which is in the datum set of f . Generally this value is produced by $\omega\text{RoundToPrecision}$ and $\omega\text{Saturate}$, or the argument is known to be in the datum set, for example, the absolute value of a number already in the set. From this precondition, it follows that $S \in \mathbb{N}$, and that X is finite in finite formats.

4.8 Decoding and encoding for external formats

The $\omega\text{DecodeExternal}$ and $\omega\text{EncodeExternal}$ functions convert between external formats and closed extended reals.

4.8.1 Decoding

The function $\omega\text{DecodeExternal}$ converts a floating-point value in an external format to a datum in the closed extended reals.

Signature

$$\omega\text{DecodeExternal}_f(x) \rightarrow X$$

Parameters

f : format of x , in $\{\text{binary64}, \text{binary32}, \text{binary16}, \text{BFloat16}\}$

Operands

x : floating-point value, in format f

Result

X : floating-point datum, a closed extended real value in $\mathcal{D}_f$, the datum set of f

Behavior

$\omega\text{DecodeExternal}(\text{Any NaN}) \rightarrow \text{NaN}$
 $\omega\text{DecodeExternal}(-\text{Inf}) \rightarrow -\infty$
 $\omega\text{DecodeExternal}(\text{Inf}) \rightarrow +\infty$
 $\omega\text{DecodeExternal}(-0) \rightarrow 0$
 $\omega\text{DecodeExternal}(x) \rightarrow$ The extended real value encoded by x

4.8.2 Encoding

Encode floating-point datum to a floating-point value in external format f .

Signature

$$\omega\text{EncodeExternal}_f(X) \rightarrow r$$

Parameters

f : target format, in $\{\text{binary64}, \text{binary32}, \text{binary16}, \text{BFloat16}\}$

Operands

X : floating-point datum, a closed extended real value in the datum set of format f

Result

r : floating-point value, in format f

Behavior

$\omega\text{EncodeExternal}(\text{NaN}) \rightarrow$ A quiet NaN, which should be the quiet NaN with zero payload and sign bit zero
 $\omega\text{EncodeExternal}(0) \rightarrow$ The code in f that decodes to the non-negative 0
 $\omega\text{EncodeExternal}(X) \rightarrow$ The code in f that decodes to X

NOTE— The $\omega\text{EncodeExternal}$ function is only called with arguments in the datum set of format f , therefore the encoding is unambiguous and independent of rounding mode.

4.9 Converting between formats

4.9.1 Conversion

Convert a floating-point value to another format, with projection specification.

Signature

$\text{Convert}_{f_x, f_r, \rho}(x) \rightarrow r$

Parameters

f_x : format of operand x

f_r : format of result r

ρ : projection specification

Operands

x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$\omega\text{Convert}(X) \rightarrow X$

$\text{Convert}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Convert}(\omega\text{Decode}_{f_x}(x)))$

4.10 Arithmetic operations

Arithmetic operations that take one or more floating-point values as operands and return one or more floating-point values are defined in this section.

4.10.1 Absolute value, Negation

Signature

$$\begin{aligned} \text{Abs}_{f_x, f_r, \rho}(x) &\rightarrow r \\ \text{Negate}_{f_x, f_r, \rho}(x) &\rightarrow r \end{aligned}$$

Parameters

f_x : format of operand x
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$$\begin{aligned} \omega\text{Abs}(\text{NaN}) &\rightarrow \text{NaN} \\ \omega\text{Abs}(-\infty) &\rightarrow +\infty \\ \omega\text{Abs}(+\infty) &\rightarrow +\infty \\ \omega\text{Abs}(X) &\rightarrow |X| \\ \text{Abs}(x) &\rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Abs}(\omega\text{Decode}_{f_x}(x))) \end{aligned}$$

$$\begin{aligned} \omega\text{Negate}(\text{NaN}) &\rightarrow \text{NaN} \\ \omega\text{Negate}(-\infty) &\rightarrow +\infty \\ \omega\text{Negate}(+\infty) &\rightarrow -\infty \\ \omega\text{Negate}(X) &\rightarrow -X \end{aligned}$$

$$\text{Negate}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Negate}(\omega\text{Decode}_{f_x}(x)))$$

4.10.2 Copying the sign

Copies the sign of a floating-point value onto another floating-point value.

Signature

$$\text{CopySign}_{f_x, f_y, f_r, \rho}(x, y) \rightarrow r$$

Parameters

f_x : format of operand x
 f_y : format of operand y
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x
 y : floating-point value, format f_y

Result

r : floating-point value, format f_r

Behavior

$\omega\text{CopySign}(\text{NaN}, *) \rightarrow \text{NaN}$
 $\omega\text{CopySign}(*, \text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{CopySign}(\pm\infty, +\infty) \rightarrow +\infty$
 $\omega\text{CopySign}(\pm\infty, -\infty) \rightarrow -\infty$
 $\omega\text{CopySign}(\pm\infty, Y) \text{ if } Y \geq 0 \rightarrow +\infty$
 $\omega\text{CopySign}(\pm\infty, Y) \text{ if } Y < 0 \rightarrow -\infty$
 $\omega\text{CopySign}(X, +\infty) \rightarrow |X|$
 $\omega\text{CopySign}(X, -\infty) \rightarrow -|X|$
 $\omega\text{CopySign}(X, Y) \text{ if } Y \geq 0 \rightarrow |X|$
 $\omega\text{CopySign}(X, Y) \text{ if } Y < 0 \rightarrow -|X|$
 $\text{CopySign}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{CopySign}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

4.10.3 Addition, Subtraction

Addition and subtraction of two floating-point values, returning a floating-point value.

Signature

$\text{Add}_{f_x, f_y, f_r, \rho}(x, y) \rightarrow r$
 $\text{Subtract}_{f_x, f_y, f_r, \rho}(x, y) \rightarrow r$

Parameters

f_x : format of operand x
 f_y : format of operand y
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x
 y : floating-point value, format f_y

Result

r : floating-point value, format f_r .

Behavior

$\omega\text{Add}(\text{NaN}, *) \rightarrow \text{NaN}$
 $\omega\text{Add}(*, \text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{Add}(+\infty, -\infty) \rightarrow \text{NaN}$
 $\omega\text{Add}(-\infty, +\infty) \rightarrow \text{NaN}$
 $\omega\text{Add}(+\infty, *) \rightarrow +\infty$
 $\omega\text{Add}(*, +\infty) \rightarrow +\infty$
 $\omega\text{Add}(-\infty, *) \rightarrow -\infty$
 $\omega\text{Add}(*, -\infty) \rightarrow -\infty$
 $\omega\text{Add}(X, Y) \rightarrow X + Y$

$\text{Add}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Add}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

$\omega\text{Subtract}(\text{NaN}, *) \rightarrow \text{NaN}$
 $\omega\text{Subtract}(*, \text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{Subtract}(+\infty, +\infty) \rightarrow \text{NaN}$
 $\omega\text{Subtract}(-\infty, -\infty) \rightarrow \text{NaN}$
 $\omega\text{Subtract}(*, +\infty) \rightarrow -\infty$
 $\omega\text{Subtract}(+\infty, *) \rightarrow +\infty$
 $\omega\text{Subtract}(*, -\infty) \rightarrow +\infty$
 $\omega\text{Subtract}(-\infty, *) \rightarrow -\infty$
 $\omega\text{Subtract}(X, Y) \rightarrow X - Y$

$\text{Subtract}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Subtract}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

4.10.4 Multiplication

Multiplication of two floating-point values, returning a floating-point value.

Signature

$\text{Multiply}_{f_x, f_y, f_r, \rho}(x, y) \rightarrow r$

Parameters

f_x : format of operand x
 f_y : format of operand y
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x
 y : floating-point value, format f_y

Result

r : floating-point value, format f_r .

Behavior

$\omega\text{Multiply}(\text{NaN}, *) \rightarrow \text{NaN}$
 $\omega\text{Multiply}(*, \text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{Multiply}(+\infty, +\infty) \rightarrow +\infty$
 $\omega\text{Multiply}(-\infty, -\infty) \rightarrow +\infty$
 $\omega\text{Multiply}(-\infty, +\infty) \rightarrow -\infty$
 $\omega\text{Multiply}(+\infty, -\infty) \rightarrow -\infty$
 $\omega\text{Multiply}(+\infty, Y) \text{ if } Y > 0 \rightarrow +\infty$
 $\omega\text{Multiply}(+\infty, Y) \text{ if } Y = 0 \rightarrow \text{NaN}$
 $\omega\text{Multiply}(+\infty, Y) \text{ if } Y < 0 \rightarrow -\infty$
 $\omega\text{Multiply}(X, +\infty) \text{ if } X > 0 \rightarrow +\infty$
 $\omega\text{Multiply}(X, +\infty) \text{ if } X = 0 \rightarrow \text{NaN}$
 $\omega\text{Multiply}(X, +\infty) \text{ if } X < 0 \rightarrow -\infty$
 $\omega\text{Multiply}(-\infty, Y) \text{ if } Y > 0 \rightarrow -\infty$
 $\omega\text{Multiply}(-\infty, Y) \text{ if } Y = 0 \rightarrow \text{NaN}$
 $\omega\text{Multiply}(-\infty, Y) \text{ if } Y < 0 \rightarrow +\infty$
 $\omega\text{Multiply}(X, -\infty) \text{ if } X > 0 \rightarrow -\infty$
 $\omega\text{Multiply}(X, -\infty) \text{ if } X = 0 \rightarrow \text{NaN}$
 $\omega\text{Multiply}(X, -\infty) \text{ if } X < 0 \rightarrow +\infty$
 $\omega\text{Multiply}(X, Y) \rightarrow X \times Y$

$\text{Multiply}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Multiply}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

4.10.5 Division

Division of two floating-point values, returning a floating-point value.

Signature

$\text{Divide}_{f_x, f_y, f_r, \rho}(x, y) \rightarrow r$

Parameters

f_x : format of operand x
 f_y : format of operand y
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x
 y : floating-point value, format f_y

Result

r : floating-point value, format f_r

Behavior

$\omega\text{Divide}(\text{NaN}, *) \rightarrow \text{NaN}$
 $\omega\text{Divide}(*, \text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{Divide}(+\infty, \pm\infty) \rightarrow \text{NaN}$
 $\omega\text{Divide}(-\infty, \pm\infty) \rightarrow \text{NaN}$
 $\omega\text{Divide}(*, 0) \rightarrow \text{NaN}$
 $\omega\text{Divide}(+\infty, Y) \text{ if } Y > 0 \rightarrow +\infty$
 $\omega\text{Divide}(+\infty, Y) \text{ if } Y < 0 \rightarrow -\infty$
 $\omega\text{Divide}(-\infty, Y) \text{ if } Y > 0 \rightarrow -\infty$
 $\omega\text{Divide}(-\infty, Y) \text{ if } Y < 0 \rightarrow +\infty$
 $\omega\text{Divide}(*, \pm\infty) \rightarrow 0$
 $\omega\text{Divide}(X, Y) \rightarrow X/Y$

$\text{Divide}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Divide}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

NOTE 1— $\text{Divide}(x, \pm\text{Inf})$ where x is finite yields 0.

NOTE 2— $\text{Divide}(x, 0)$ yields NaN. Returning Inf for finite x would imply $1/(1/-\infty) \rightarrow \infty$, an inconsistency.

4.10.6 Fused Multiply–Add

Compute $R = X \times Y + Z$, with computation in the reals. Rounding and the determination of overflow and underflow are applied only on the result. The behavior of $\omega\text{FMA}(X, Y, Z)$ is equivalent [12] to the behavior of $\omega\text{Add}(\omega\text{Multiply}(X, Y), Z)$.

Signature

$$\text{FMA}_{f_x, f_y, f_z, f_r, \rho}(x, y, z) \rightarrow r$$

Parameters

f_x : format of operand x
 f_y : format of operand y
 f_z : format of operand z
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x
 y : floating-point value, format f_y
 z : floating-point value, format f_z

Result

r : floating-point value, format f_r

Behavior

$\omega\text{FMA}(\text{NaN}, *, *) \rightarrow \text{NaN}$
 $\omega\text{FMA}(*, \text{NaN}, *) \rightarrow \text{NaN}$
 $\omega\text{FMA}(*, *, \text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{FMA}(0, \pm\infty, *) \rightarrow \text{NaN}$
 $\omega\text{FMA}(\pm\infty, 0, *) \rightarrow \text{NaN}$
 $\omega\text{FMA}(X, +\infty, +\infty) \text{ if } X < 0 \rightarrow \text{NaN}$
 $\omega\text{FMA}(X, -\infty, +\infty) \text{ if } X > 0 \rightarrow \text{NaN}$
 $\omega\text{FMA}(+\infty, Y, +\infty) \text{ if } Y < 0 \rightarrow \text{NaN}$
 $\omega\text{FMA}(-\infty, Y, +\infty) \text{ if } Y > 0 \rightarrow \text{NaN}$
 $\omega\text{FMA}(X, -\infty, -\infty) \text{ if } X < 0 \rightarrow \text{NaN}$
 $\omega\text{FMA}(X, +\infty, -\infty) \text{ if } X > 0 \rightarrow \text{NaN}$
 $\omega\text{FMA}(-\infty, Y, -\infty) \text{ if } Y < 0 \rightarrow \text{NaN}$
 $\omega\text{FMA}(+\infty, Y, -\infty) \text{ if } Y > 0 \rightarrow \text{NaN}$
 $\omega\text{FMA}(-\infty, +\infty, +\infty) \rightarrow \text{NaN}$
 $\omega\text{FMA}(+\infty, -\infty, +\infty) \rightarrow \text{NaN}$
 $\omega\text{FMA}(+\infty, +\infty, -\infty) \rightarrow \text{NaN}$
 $\omega\text{FMA}(-\infty, -\infty, -\infty) \rightarrow \text{NaN}$
 $\omega\text{FMA}(+\infty, +\infty, *) \rightarrow +\infty$
 $\omega\text{FMA}(-\infty, -\infty, *) \rightarrow +\infty$
 $\omega\text{FMA}(+\infty, -\infty, *) \rightarrow -\infty$
 $\omega\text{FMA}(-\infty, +\infty, *) \rightarrow -\infty$
 $\omega\text{FMA}(*, *, +\infty) \rightarrow +\infty$
 $\omega\text{FMA}(*, *, -\infty) \rightarrow -\infty$
 $\omega\text{FMA}(X, -\infty, *) \text{ if } X > 0 \rightarrow -\infty$
 $\omega\text{FMA}(X, -\infty, *) \text{ if } X < 0 \rightarrow +\infty$
 $\omega\text{FMA}(X, +\infty, *) \text{ if } X > 0 \rightarrow +\infty$
 $\omega\text{FMA}(X, +\infty, *) \text{ if } X < 0 \rightarrow -\infty$
 $\omega\text{FMA}(+\infty, Y, *) \text{ if } Y > 0 \rightarrow +\infty$
 $\omega\text{FMA}(+\infty, Y, *) \text{ if } Y < 0 \rightarrow -\infty$
 $\omega\text{FMA}(-\infty, Y, *) \text{ if } Y > 0 \rightarrow -\infty$
 $\omega\text{FMA}(-\infty, Y, *) \text{ if } Y < 0 \rightarrow +\infty$
 $\omega\text{FMA}(X, Y, Z) \rightarrow X \times Y + Z$

$$\text{FMA}(x, y, z) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{FMA}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y), \omega\text{Decode}_{f_z}(z)))$$

4.10.7 Fused Add–Add

Compute $R = X + Y + Z$, with computation in the reals. Rounding and the determination of overflow and underflow are applied only on the result. The behavior of $\omega\text{FAA}(X, Y, Z)$ is equivalent [12] to the behavior of $\omega\text{Add}(\omega\text{Add}(X, Y), Z)$, which is in turn equivalent to $\omega\text{Add}(X, \omega\text{Add}(Y, Z))$.

Signature

$$\text{FAA}_{f_x, f_y, f_z, f_r, \rho}(x, y, z) \rightarrow r$$

Parameters

f_x : format of operand x
 f_y : format of operand y
 f_z : format of operand z
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x
 y : floating-point value, format f_y
 z : floating-point value, format f_z

Result

r : floating-point value, format f_r

Behavior

$\omega\text{FAA}(\text{NaN}, *, *) \rightarrow \text{NaN}$
 $\omega\text{FAA}(*, \text{NaN}, *) \rightarrow \text{NaN}$
 $\omega\text{FAA}(*, *, \text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{FAA}(+\infty, -\infty, *) \rightarrow \text{NaN}$
 $\omega\text{FAA}(-\infty, +\infty, *) \rightarrow \text{NaN}$
 $\omega\text{FAA}(+\infty, *, -\infty) \rightarrow \text{NaN}$
 $\omega\text{FAA}(-\infty, *, +\infty) \rightarrow \text{NaN}$
 $\omega\text{FAA}(*, +\infty, -\infty) \rightarrow \text{NaN}$
 $\omega\text{FAA}(*, -\infty, +\infty) \rightarrow \text{NaN}$
 $\omega\text{FAA}(+\infty, +\infty, +\infty) \rightarrow +\infty$
 $\omega\text{FAA}(-\infty, -\infty, -\infty) \rightarrow -\infty$
 $\omega\text{FAA}(*, +\infty, +\infty) \rightarrow +\infty$
 $\omega\text{FAA}(+\infty, *, +\infty) \rightarrow +\infty$
 $\omega\text{FAA}(+\infty, +\infty, *) \rightarrow +\infty$
 $\omega\text{FAA}(*, -\infty, -\infty) \rightarrow -\infty$
 $\omega\text{FAA}(-\infty, *, -\infty) \rightarrow -\infty$
 $\omega\text{FAA}(-\infty, -\infty, *) \rightarrow -\infty$
 $\omega\text{FAA}(+\infty, *, *) \rightarrow +\infty$
 $\omega\text{FAA}(*, +\infty, *) \rightarrow +\infty$
 $\omega\text{FAA}(*, *, +\infty) \rightarrow +\infty$
 $\omega\text{FAA}(-\infty, *, *) \rightarrow -\infty$
 $\omega\text{FAA}(*, -\infty, *) \rightarrow -\infty$
 $\omega\text{FAA}(*, *, -\infty) \rightarrow -\infty$
 $\omega\text{FAA}(X, Y, Z) \rightarrow X + Y + Z$
 $\text{FAA}(x, y, z) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{FAA}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y), \omega\text{Decode}_{f_z}(z)))$

4.10.8 Square root, Reciprocal, Reciprocal square root

Signature

$\text{Sqrt}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{Recip}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{RSqrt}_{f_x, f_r, \rho}(x) \rightarrow r$

Parameters

f_x : format of operand x
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$\omega\text{Sqrt}(\text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{Sqrt}(-\infty) \rightarrow \text{NaN}$
 $\omega\text{Sqrt}(X) \text{ if } X < 0 \rightarrow \text{NaN}$
 $\omega\text{Sqrt}(+\infty) \rightarrow +\infty$
 $\omega\text{Sqrt}(X) \rightarrow \sqrt{X}$
 $\text{Sqrt}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Sqrt}(\omega\text{Decode}_{f_x}(x)))$

$\omega\text{Recip}(\text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{Recip}(0) \rightarrow \text{NaN}$
 $\omega\text{Recip}(\pm\infty) \rightarrow 0$
 $\omega\text{Recip}(X) \rightarrow 1/X$
 $\text{Recip}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Recip}(\omega\text{Decode}_{f_x}(x)))$

$\omega\text{RSqrt}(\text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{RSqrt}(-\infty) \rightarrow \text{NaN}$
 $\omega\text{RSqrt}(X) \text{ if } X \leq 0 \rightarrow \text{NaN}$
 $\omega\text{RSqrt}(+\infty) \rightarrow 0$
 $\omega\text{RSqrt}(X) \rightarrow 1/\sqrt{X}$
 $\text{RSqrt}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{RSqrt}(\omega\text{Decode}_{f_x}(x)))$

4.10.9 Logarithm, Exponentiation

Signature

$\text{Exp}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{Exp2}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{Log}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{Log2}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{LogOnePlus}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{ExpMinusOne}_{f_x, f_r, \rho}(x) \rightarrow r$

Parameters

f_x : format of operand x
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$\omega\text{Exp}(\text{NaN}) \rightarrow \text{NaN}$	$\omega\text{Exp2}(\text{NaN}) \rightarrow \text{NaN}$
$\omega\text{Exp}(+\infty) \rightarrow +\infty$	$\omega\text{Exp2}(+\infty) \rightarrow +\infty$
$\omega\text{Exp}(-\infty) \rightarrow 0$	$\omega\text{Exp2}(-\infty) \rightarrow 0$
$\omega\text{Exp}(X) \rightarrow e^X$	$\omega\text{Exp2}(X) \rightarrow 2^X$
$\text{Exp}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Exp}(\omega\text{Decode}_{f_x}(x)))$	$\text{Exp2}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Exp2}(\omega\text{Decode}_{f_x}(x)))$
$\omega\text{Log}(\text{NaN}) \rightarrow \text{NaN}$	$\omega\text{Log2}(\text{NaN}) \rightarrow \text{NaN}$
$\omega\text{Log}(-\infty) \rightarrow \text{NaN}$	$\omega\text{Log2}(-\infty) \rightarrow \text{NaN}$
$\omega\text{Log}(+\infty) \rightarrow +\infty$	$\omega\text{Log2}(+\infty) \rightarrow +\infty$
$\omega\text{Log}(X) \text{ if } X < 0 \rightarrow \text{NaN}$	$\omega\text{Log2}(X) \text{ if } X < 0 \rightarrow \text{NaN}$
$\omega\text{Log}(0) \rightarrow -\infty$	$\omega\text{Log2}(0) \rightarrow -\infty$
$\omega\text{Log}(X) \rightarrow \log_e X$	$\omega\text{Log2}(X) \rightarrow \log_2 X$
$\text{Log}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Log}(\omega\text{Decode}_{f_x}(x)))$	$\text{Log2}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Log2}(\omega\text{Decode}_{f_x}(x)))$
$\omega\text{LogOnePlus}(X) \rightarrow \omega\text{Log}(\omega\text{Add}(1, X))$	
$\text{LogOnePlus}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{LogOnePlus}(\omega\text{Decode}_{f_x}(x)))$	
$\omega\text{ExpMinusOne}(X) \rightarrow \omega\text{Subtract}(\omega\text{Exp}(X), 1)$	
$\text{ExpMinusOne}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ExpMinusOne}(\omega\text{Decode}_{f_x}(x)))$	

4.10.10 Trigonometric functions

Signature

$$\text{Sin}_{f_x, f_r, \rho}(x) \rightarrow r$$

$$\text{Cos}_{f_x, f_r, \rho}(x) \rightarrow r$$

$$\text{Tan}_{f_x, f_r, \rho}(x) \rightarrow r$$

$$\text{ArcSin}_{f_x, f_r, \rho}(x) \rightarrow r$$

$$\text{ArcCos}_{f_x, f_r, \rho}(x) \rightarrow r$$

$$\text{ArcTan}_{f_x, f_r, \rho}(x) \rightarrow r$$

Parameters

f_x : format of operand x

f_r : format of result r

ρ : projection specification

Operands

x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$$\omega\text{Sin}(\text{NaN}) \rightarrow \text{NaN}$$

$$\omega\text{Sin}(+\infty) \rightarrow \text{NaN}$$

$$\omega\text{Sin}(-\infty) \rightarrow \text{NaN}$$

$$\omega\text{Sin}(X) \rightarrow \sin X$$

$$\text{Sin}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Sin}(\omega\text{Decode}_{f_x}(x)))$$

$$\omega\text{ArcSin}(\text{NaN}) \rightarrow \text{NaN}$$

$$\omega\text{ArcSin}(+\infty) \rightarrow \text{NaN}$$

$$\omega\text{ArcSin}(-\infty) \rightarrow \text{NaN}$$

$$\omega\text{ArcSin}(X) \text{ if } X < -1 \text{ or } X > 1 \rightarrow \text{NaN}$$

$$\omega\text{ArcSin}(X) \rightarrow \arcsin X$$

$$\text{ArcSin}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcSin}(\omega\text{Decode}_{f_x}(x)))$$

$$\omega\text{Cos}(\text{NaN}) \rightarrow \text{NaN}$$

$$\omega\text{Cos}(+\infty) \rightarrow \text{NaN}$$

$$\omega\text{Cos}(-\infty) \rightarrow \text{NaN}$$

$$\omega\text{Cos}(X) \rightarrow \cos X$$

$$\text{Cos}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Cos}(\omega\text{Decode}_{f_x}(x)))$$

$$\omega\text{ArcCos}(\text{NaN}) \rightarrow \text{NaN}$$

$$\omega\text{ArcCos}(+\infty) \rightarrow \text{NaN}$$

$$\omega\text{ArcCos}(-\infty) \rightarrow \text{NaN}$$

$$\omega\text{ArcCos}(X) \text{ if } X < -1 \text{ or } X > 1 \rightarrow \text{NaN}$$

$$\omega\text{ArcCos}(X) \rightarrow \arccos X$$

$$\text{ArcCos}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcCos}(\omega\text{Decode}_{f_x}(x)))$$

$$\omega\text{Tan}(\text{NaN}) \rightarrow \text{NaN}$$

$$\omega\text{Tan}(+\infty) \rightarrow \text{NaN}$$

$$\omega\text{Tan}(-\infty) \rightarrow \text{NaN}$$

$$\omega\text{Tan}(X) \text{ if } \cos X = 0 \text{ and } \sin X > 0 \rightarrow +\infty$$

$$\omega\text{Tan}(X) \text{ if } \cos X = 0 \text{ and } \sin X < 0 \rightarrow -\infty$$

$$\omega\text{Tan}(X) \rightarrow \tan X$$

$$\text{Tan}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Tan}(\omega\text{Decode}_{f_x}(x)))$$

$$\omega\text{ArcTan}(\text{NaN}) \rightarrow \text{NaN}$$

$$\omega\text{ArcTan}(+\infty) \rightarrow \frac{\pi}{2}$$

$$\omega\text{ArcTan}(-\infty) \rightarrow -\frac{\pi}{2}$$

$$\omega\text{ArcTan}(X) \rightarrow \arctan X$$

$$\text{ArcTan}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcTan}(\omega\text{Decode}_{f_x}(x)))$$

NOTE— There are no datums X in the datum set of any format f for which $\cos X = 0$, as all floats are rational, but the special case is included for documentation, and to support formal verification.

4.10.11 Hyperbolic functions

Signature

$\text{Sinh}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{Cosh}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{Tanh}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{ArcSinh}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{ArcCosh}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{ArcTanh}_{f_x, f_r, \rho}(x) \rightarrow r$

Parameters

f_x : format of operand x
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$\omega\text{Sinh}(\text{NaN}) \rightarrow \text{NaN}$	$\omega\text{ArcSinh}(\text{NaN}) \rightarrow \text{NaN}$
$\omega\text{Sinh}(+\infty) \rightarrow +\infty$	$\omega\text{ArcSinh}(+\infty) \rightarrow +\infty$
$\omega\text{Sinh}(-\infty) \rightarrow -\infty$	$\omega\text{ArcSinh}(-\infty) \rightarrow -\infty$
$\omega\text{Sinh}(X) \rightarrow \sinh X$	$\omega\text{ArcSinh}(X) \rightarrow \operatorname{arcsinh} X$
$\text{Sinh}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Sinh}(\omega\text{Decode}_{f_x}(x)))$	$\text{ArcSinh}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcSinh}(\omega\text{Decode}_{f_x}(x)))$
$\omega\text{Cosh}(\text{NaN}) \rightarrow \text{NaN}$	$\omega\text{ArcCosh}(\text{NaN}) \rightarrow \text{NaN}$
$\omega\text{Cosh}(+\infty) \rightarrow +\infty$	$\omega\text{ArcCosh}(+\infty) \rightarrow +\infty$
$\omega\text{Cosh}(-\infty) \rightarrow +\infty$	$\omega\text{ArcCosh}(-\infty) \rightarrow \text{NaN}$
$\omega\text{Cosh}(X) \rightarrow \cosh X$	$\omega\text{ArcCosh}(X) \text{ if } X < 1 \rightarrow \text{NaN}$
$\text{Cosh}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Cosh}(\omega\text{Decode}_{f_x}(x)))$	$\omega\text{ArcCosh}(X) \rightarrow \operatorname{arccosh} X$
	$\text{ArcCosh}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcCosh}(\omega\text{Decode}_{f_x}(x)))$
$\omega\text{Tanh}(\text{NaN}) \rightarrow \text{NaN}$	$\omega\text{ArcTanh}(\text{NaN}) \rightarrow \text{NaN}$
$\omega\text{Tanh}(+\infty) \rightarrow 1$	$\omega\text{ArcTanh}(+\infty) \rightarrow \text{NaN}$
$\omega\text{Tanh}(-\infty) \rightarrow -1$	$\omega\text{ArcTanh}(-\infty) \rightarrow \text{NaN}$
$\omega\text{Tanh}(X) \rightarrow \tanh X$	$\omega\text{ArcTanh}(1) \rightarrow +\infty$
$\text{Tanh}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Tanh}(\omega\text{Decode}_{f_x}(x)))$	$\omega\text{ArcTanh}(-1) \rightarrow -\infty$
	$\omega\text{ArcTanh}(X) \text{ if } X < -1 \text{ or } X > 1 \rightarrow \text{NaN}$
	$\omega\text{ArcTanh}(X) \rightarrow \operatorname{arctanh} X$
	$\text{ArcTanh}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcTanh}(\omega\text{Decode}_{f_x}(x)))$

4.10.12 Trigonometric π -functions

Signature

$\text{SinPi}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{CosPi}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{TanPi}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{ArcSinPi}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{ArcCosPi}_{f_x, f_r, \rho}(x) \rightarrow r$
 $\text{ArcTanPi}_{f_x, f_r, \rho}(x) \rightarrow r$

Parameters

f_x : format of operand x
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$\omega\text{SinPi}(\text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{SinPi}(+\infty) \rightarrow \text{NaN}$
 $\omega\text{SinPi}(-\infty) \rightarrow \text{NaN}$
 $\omega\text{SinPi}(X) \rightarrow \sin(X \times \pi)$
 $\text{SinPi}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{SinPi}(\omega\text{Decode}_{f_x}(x)))$

$\omega\text{CosPi}(\text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{CosPi}(+\infty) \rightarrow \text{NaN}$
 $\omega\text{CosPi}(-\infty) \rightarrow \text{NaN}$
 $\omega\text{CosPi}(X) \rightarrow \cos(X \times \pi)$
 $\text{CosPi}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{CosPi}(\omega\text{Decode}_{f_x}(x)))$

$\omega\text{TanPi}(\text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{TanPi}(+\infty) \rightarrow \text{NaN}$
 $\omega\text{TanPi}(-\infty) \rightarrow \text{NaN}$
 $\omega\text{TanPi}(X) \text{ if } \cos(X \times \pi) = 0 \text{ and } \sin(X \times \pi) > 0 \rightarrow +\infty$
 $\omega\text{TanPi}(X) \text{ if } \cos(X \times \pi) = 0 \text{ and } \sin(X \times \pi) < 0 \rightarrow -\infty$
 $\omega\text{TanPi}(X) \rightarrow \tan(X \times \pi)$
 $\text{TanPi}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{TanPi}(\omega\text{Decode}_{f_x}(x)))$

$\omega\text{ArcSinPi}(\text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{ArcSinPi}(+\infty) \rightarrow \text{NaN}$
 $\omega\text{ArcSinPi}(-\infty) \rightarrow \text{NaN}$
 $\omega\text{ArcSinPi}(X) \text{ if } X < -1 \text{ or } X > 1 \rightarrow \text{NaN}$
 $\omega\text{ArcSinPi}(X) \rightarrow (\arcsin X)/\pi$
 $\text{ArcSinPi}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcSinPi}(\omega\text{Decode}_{f_x}(x)))$
 $\omega\text{ArcCosPi}(\text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{ArcCosPi}(+\infty) \rightarrow \text{NaN}$
 $\omega\text{ArcCosPi}(-\infty) \rightarrow \text{NaN}$
 $\omega\text{ArcCosPi}(X) \text{ if } X < -1 \text{ or } X > 1 \rightarrow \text{NaN}$
 $\omega\text{ArcCosPi}(X) \rightarrow (\arccos X)/\pi$
 $\text{ArcCosPi}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcCosPi}(\omega\text{Decode}_{f_x}(x)))$

$\omega\text{ArcTanPi}(\text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{ArcTanPi}(+\infty) \rightarrow 1/2$
 $\omega\text{ArcTanPi}(-\infty) \rightarrow -1/2$
 $\omega\text{ArcTanPi}(X) \rightarrow (\arctan X)/\pi$
 $\text{ArcTanPi}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcTanPi}(\omega\text{Decode}_{f_x}(x)))$

4.10.13 Softplus

Signature

$\text{Softplus}_{f_x, f_r, \rho}(x) \rightarrow r$

Parameters

f_x : format of operand x
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$\omega\text{Softplus}(\text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{Softplus}(+\infty) \rightarrow +\infty$
 $\omega\text{Softplus}(-\infty) \rightarrow 0$
 $\omega\text{Softplus}(X) \rightarrow \log_e(1 + e^X)$
 $\text{Softplus}(x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Softplus}(\omega\text{Decode}_{f_x}(x)))$

4.10.14 Hypotenuse

Signature

$\text{Hypot}_{f_x, f_y, f_r, \rho}(x, y) \rightarrow r$

Parameters

f_x : format of operand x
 f_y : format of operand y
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x
 y : floating-point value, format f_y

Result

r : floating-point value, format f_r

Behavior

$\omega\text{Hypot}(\text{NaN}, *) \rightarrow \text{NaN}$

$\omega\text{Hypot}(*, \text{NaN}) \rightarrow \text{NaN}$

$\omega\text{Hypot}(*, \pm\infty) \rightarrow +\infty$

$\omega\text{Hypot}(\pm\infty, *) \rightarrow +\infty$

$\omega\text{Hypot}(X, Y) \rightarrow \sqrt{|X|^2 + |Y|^2}$

$\text{Hypot}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Hypot}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

4.10.15 Inverse tangent of two variables

Signature

$\text{ArcTan2}_{f_y, f_x, f_r, \rho}(y, x) \rightarrow r$

Parameters

f_y : format of operand y
 f_x : format of operand x
 f_r : format of result r
 ρ : projection specification

Operands

y : floating-point value, format f_y
 x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$\omega\text{ArcTan2}(\text{NaN}, *) \rightarrow \text{NaN}$
 $\omega\text{ArcTan2}(*, \text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{ArcTan2}(0, 0) \rightarrow \text{NaN}$
 $\omega\text{ArcTan2}(-\infty, \pm\infty) \rightarrow \text{NaN}$
 $\omega\text{ArcTan2}(+\infty, \pm\infty) \rightarrow \text{NaN}$
 $\omega\text{ArcTan2}(*, +\infty) \rightarrow 0$
 $\omega\text{ArcTan2}(0, X) \text{ if } X > 0 \rightarrow 0$
 $\omega\text{ArcTan2}(+\infty, *) \rightarrow \frac{\pi}{2}$
 $\omega\text{ArcTan2}(Y, 0) \text{ if } Y > 0 \rightarrow \frac{\pi}{2}$
 $\omega\text{ArcTan2}(Y, -\infty) \text{ if } Y \geq 0 \rightarrow \pi$
 $\omega\text{ArcTan2}(Y, -\infty) \text{ if } Y < 0 \rightarrow -\pi$
 $\omega\text{ArcTan2}(0, X) \text{ if } X < 0 \rightarrow \pi$
 $\omega\text{ArcTan2}(-\infty, *) \rightarrow -\frac{\pi}{2}$
 $\omega\text{ArcTan2}(Y, 0) \text{ if } Y < 0 \rightarrow -\frac{\pi}{2}$
 $\omega\text{ArcTan2}(Y, X) \text{ if } X > 0 \rightarrow \arctan(Y/X)$
 $\omega\text{ArcTan2}(Y, X) \text{ if } Y > 0 \text{ and } X < 0 \rightarrow \arctan(Y/X) + \pi$
 $\omega\text{ArcTan2}(Y, X) \text{ if } Y < 0 \text{ and } X < 0 \rightarrow \arctan(Y/X) - \pi$
 $\text{ArcTan2}(y, x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcTan2}(\omega\text{Decode}_{f_y}(y), \omega\text{Decode}_{f_x}(x)))$

NOTE 1—It would be inconsistent to define $\text{ArcTan2}(0, 0)$ as 0, as the limit is indeterminate.

NOTE 2— $\text{ArcTan2}(\pm\text{Inf}, \pm\text{Inf})$ yields NaN, which is consistent with $\text{Divide}(\pm\text{Inf}, \pm\text{Inf})$.

4.10.16 Inverse tangent of two variables (π -variant)

Signature

$\text{ArcTan2Pi}_{f_y, f_x, f_r, \rho}(y, x) \rightarrow r$

Parameters

f_y : format of operand y
 f_x : format of operand x
 f_r : format of result r
 ρ : projection specification

Operands

y : floating-point value, format f_y
 x : floating-point value, format f_x

Result

r : floating-point value, format f_r

Behavior

$\omega\text{ArcTan2Pi}(\text{NaN}, *) \rightarrow \text{NaN}$
 $\omega\text{ArcTan2Pi}(*, \text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{ArcTan2Pi}(0, 0) \rightarrow \text{NaN}$
 $\omega\text{ArcTan2Pi}(-\infty, \pm\infty) \rightarrow \text{NaN}$
 $\omega\text{ArcTan2Pi}(+\infty, \pm\infty) \rightarrow \text{NaN}$
 $\omega\text{ArcTan2Pi}(*, +\infty) \rightarrow 0$
 $\omega\text{ArcTan2Pi}(0, X) \text{ if } X > 0 \rightarrow 0$
 $\omega\text{ArcTan2Pi}(+\infty, *) \rightarrow 1/2$
 $\omega\text{ArcTan2Pi}(Y, 0) \text{ if } Y > 0 \rightarrow 1/2$
 $\omega\text{ArcTan2Pi}(Y, -\infty) \text{ if } Y \geq 0 \rightarrow 1$
 $\omega\text{ArcTan2Pi}(Y, -\infty) \text{ if } Y < 0 \rightarrow -1$
 $\omega\text{ArcTan2Pi}(0, X) \text{ if } X < 0 \rightarrow 1$
 $\omega\text{ArcTan2Pi}(-\infty, *) \rightarrow -1/2$
 $\omega\text{ArcTan2Pi}(Y, 0) \text{ if } Y < 0 \rightarrow -1/2$
 $\omega\text{ArcTan2Pi}(Y, X) \text{ if } X > 0 \rightarrow (\arctan(Y/X))/\pi$
 $\omega\text{ArcTan2Pi}(Y, X) \text{ if } Y > 0 \text{ and } X < 0 \rightarrow (\arctan(Y/X) + \pi)/\pi$
 $\omega\text{ArcTan2Pi}(Y, X) \text{ if } Y < 0 \text{ and } X < 0 \rightarrow (\arctan(Y/X) - \pi)/\pi$
 $\text{ArcTan2Pi}(y, x) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{ArcTan2Pi}(\omega\text{Decode}_{f_y}(y), \omega\text{Decode}_{f_x}(x)))$

NOTE 1—It would be inconsistent to define $\text{ArcTan2Pi}(0, 0)$ as 0, as the limit is indeterminate.

NOTE 2— $\text{ArcTan2Pi}(\pm\text{Inf}, \pm\text{Inf})$ yields NaN, which is consistent with $\text{Divide}(\pm\text{Inf}, \pm\text{Inf})$.

4.11 Extrema

4.11.1 Minimum and maximum, and number variants

Minimum and maximum, with or without propagation of NaN.

Signature

Operation $_{f_x, f_y, f_r, \rho}(x, y) \rightarrow r$

Parameters

f_x : format of operand x

f_y : format of operand y

f_r : format of result r

ρ : projection specification

Operands

x : floating-point value, format f_x

y : floating-point value, format f_y

Result

r : floating-point value, format f_r

Behavior

$\omega\text{Minimum}(\text{NaN}, *) \rightarrow \text{NaN}$

$\omega\text{Minimum}(*, \text{NaN}) \rightarrow \text{NaN}$

$\omega\text{Minimum}(+\infty, +\infty) \rightarrow +\infty$

$\omega\text{Minimum}(-\infty, -\infty) \rightarrow -\infty$

$\omega\text{Minimum}(+\infty, -\infty) \rightarrow -\infty$

$\omega\text{Minimum}(-\infty, +\infty) \rightarrow -\infty$

$\omega\text{Minimum}(+\infty, Y) \rightarrow Y$

$\omega\text{Minimum}(X, +\infty) \rightarrow X$

$\omega\text{Minimum}(-\infty, *) \rightarrow -\infty$

$\omega\text{Minimum}(*, -\infty) \rightarrow -\infty$

$\omega\text{Minimum}(X, Y) \rightarrow \text{if } X < Y \text{ then } X \text{ else } Y$

$\omega\text{Maximum}(\text{NaN}, *) \rightarrow \text{NaN}$

$\omega\text{Maximum}(*, \text{NaN}) \rightarrow \text{NaN}$

$\omega\text{Maximum}(+\infty, +\infty) \rightarrow +\infty$

$\omega\text{Maximum}(-\infty, -\infty) \rightarrow -\infty$

$\omega\text{Maximum}(+\infty, -\infty) \rightarrow +\infty$

$\omega\text{Maximum}(-\infty, +\infty) \rightarrow +\infty$

$\omega\text{Maximum}(+\infty, *) \rightarrow +\infty$

$\omega\text{Maximum}(*, +\infty) \rightarrow +\infty$

$\omega\text{Maximum}(-\infty, Y) \rightarrow Y$

$\omega\text{Maximum}(X, -\infty) \rightarrow X$

$\omega\text{Maximum}(X, Y) \rightarrow \text{if } X < Y \text{ then } Y \text{ else } X$

$\text{Minimum}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Minimum}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

$\text{Maximum}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Maximum}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

$\omega\text{MinimumNumber}(\text{NaN}, \text{NaN}) \rightarrow \text{NaN}$

$\omega\text{MinimumNumber}(X, \text{NaN}) \rightarrow X$

$\omega\text{MinimumNumber}(\text{NaN}, Y) \rightarrow Y$

$\omega\text{MinimumNumber}(X, Y) \rightarrow \omega\text{Minimum}(X, Y)$

$\omega\text{MaximumNumber}(\text{NaN}, \text{NaN}) \rightarrow \text{NaN}$

$\omega\text{MaximumNumber}(X, \text{NaN}) \rightarrow X$

$\omega\text{MaximumNumber}(\text{NaN}, Y) \rightarrow Y$

$\omega\text{MaximumNumber}(X, Y) \rightarrow \omega\text{Maximum}(X, Y)$

$\text{MinimumNumber}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{MinimumNumber}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

$\text{MaximumNumber}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{MaximumNumber}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

4.11.2 Minimum and maximum magnitude, and number variants

Minimum and maximum by magnitude, with or without propagation of NaN.

Signature

Operation _{f_x, f_y, f_r, ρ} (x, y) $\rightarrow r$

Parameters

f_x : format of operand x

f_y : format of operand y

f_r : format of result r

ρ : projection specification

Operands

x : floating-point value, format f_x

y : floating-point value, format f_y

Result

r : floating-point value, format f_r

Behavior

$\omega\text{MinimumMagnitude}(\text{NaN}, *) \rightarrow \text{NaN}$	$\omega\text{MaximumMagnitude}(\text{NaN}, *) \rightarrow \text{NaN}$
$\omega\text{MinimumMagnitude}(*, \text{NaN}) \rightarrow \text{NaN}$	$\omega\text{MaximumMagnitude}(*, \text{NaN}) \rightarrow \text{NaN}$
$\omega\text{MinimumMagnitude}(+\infty, +\infty) \rightarrow +\infty$	$\omega\text{MaximumMagnitude}(+\infty, *) \rightarrow +\infty$
$\omega\text{MinimumMagnitude}(-\infty, -\infty) \rightarrow -\infty$	$\omega\text{MaximumMagnitude}(*, +\infty) \rightarrow +\infty$
$\omega\text{MinimumMagnitude}(+\infty, -\infty) \rightarrow -\infty$	$\omega\text{MaximumMagnitude}(-\infty, *) \rightarrow -\infty$
$\omega\text{MinimumMagnitude}(-\infty, +\infty) \rightarrow +\infty$	$\omega\text{MaximumMagnitude}(*, -\infty) \rightarrow -\infty$
$\omega\text{MinimumMagnitude}(\pm\infty, Y) \rightarrow Y$	$\omega\text{MaximumMagnitude}(X, Y) \text{ if } X > Y \rightarrow X$
$\omega\text{MinimumMagnitude}(X, \pm\infty) \rightarrow X$	$\omega\text{MaximumMagnitude}(X, Y) \text{ if } X < Y \rightarrow Y$
$\omega\text{MinimumMagnitude}(X, Y) \text{ if } X < Y \rightarrow Y$	$\omega\text{MaximumMagnitude}(X, Y) \text{ if } X = Y \rightarrow$
$\omega\text{MinimumMagnitude}(X, Y) \text{ if } X > Y \rightarrow X$	$\text{if } X < Y \text{ then } Y \text{ else } X$
$\omega\text{MinimumMagnitude}(X, Y) \text{ if } X = Y \rightarrow$	
$\text{if } X < Y \text{ then } X \text{ else } Y$	

$\text{MinimumMagnitude}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{MinimumMagnitude}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

$\text{MaximumMagnitude}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{MaximumMagnitude}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

$\omega\text{MinimumMagnitudeNumber}(X, \text{NaN}) \rightarrow X$

$\omega\text{MinimumMagnitudeNumber}(\text{NaN}, Y) \rightarrow Y$

$\omega\text{MinimumMagnitudeNumber}(X, Y) \rightarrow \omega\text{MinimumMagnitude}(X, Y)$

$\text{MinimumMagnitudeNumber}(x, y) \rightarrow$

$\omega\text{Project}_{f_r, \rho}(\omega\text{MinimumMagnitudeNumber}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

$\omega\text{MaximumMagnitudeNumber}(X, \text{NaN}) \rightarrow X$

$\omega\text{MaximumMagnitudeNumber}(\text{NaN}, Y) \rightarrow Y$

$\omega\text{MaximumMagnitudeNumber}(X, Y) \rightarrow \omega\text{MaximumMagnitude}(X, Y)$

$\text{MaximumMagnitudeNumber}(x, y) \rightarrow$

$\omega\text{Project}_{f_r, \rho}(\omega\text{MaximumMagnitudeNumber}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

4.11.3 Minimum and maximum finite variants

Minimum and maximum finite value.

Signature

Operation _{f_x, f_y, f_r, ρ} (x, y) $\rightarrow r$

Parameters

f_x : format of operand x

f_y : format of operand y

f_r : format of result r

ρ : projection specification

Operands

x : floating-point value, format f_x

y : floating-point value, format f_y

Result

r : floating-point value, format f_r

Behavior

$\omega\text{MinimumFinite}(\text{NaN}, \text{NaN}) \rightarrow \text{NaN}$

$\omega\text{MinimumFinite}(\text{NaN}, Y) \rightarrow Y$

$\omega\text{MinimumFinite}(X, \text{NaN}) \rightarrow X$

$\omega\text{MinimumFinite}(+\infty, +\infty) \rightarrow +\infty$

$\omega\text{MinimumFinite}(-\infty, -\infty) \rightarrow -\infty$

$\omega\text{MinimumFinite}(+\infty, -\infty) \rightarrow -\infty$

$\omega\text{MinimumFinite}(-\infty, +\infty) \rightarrow -\infty$

$\omega\text{MinimumFinite}(+\infty, Y) \rightarrow Y$

$\omega\text{MinimumFinite}(X, +\infty) \rightarrow X$

$\omega\text{MinimumFinite}(-\infty, Y) \rightarrow Y$

$\omega\text{MinimumFinite}(X, -\infty) \rightarrow X$

$\omega\text{MinimumFinite}(X, Y) \rightarrow$

if $X < Y$ **then** X **else** Y

$\omega\text{MaximumFinite}(\text{NaN}, \text{NaN}) \rightarrow \text{NaN}$

$\omega\text{MaximumFinite}(\text{NaN}, Y) \rightarrow Y$

$\omega\text{MaximumFinite}(X, \text{NaN}) \rightarrow X$

$\omega\text{MaximumFinite}(+\infty, +\infty) \rightarrow +\infty$

$\omega\text{MaximumFinite}(-\infty, -\infty) \rightarrow -\infty$

$\omega\text{MaximumFinite}(+\infty, -\infty) \rightarrow +\infty$

$\omega\text{MaximumFinite}(-\infty, +\infty) \rightarrow +\infty$

$\omega\text{MaximumFinite}(+\infty, Y) \rightarrow Y$

$\omega\text{MaximumFinite}(X, +\infty) \rightarrow X$

$\omega\text{MaximumFinite}(-\infty, Y) \rightarrow Y$

$\omega\text{MaximumFinite}(X, -\infty) \rightarrow X$

$\omega\text{MaximumFinite}(X, Y) \rightarrow$

if $X < Y$ **then** Y **else** X

$\text{MinimumFinite}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{MinimumFinite}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

$\text{MaximumFinite}(x, y) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{MaximumFinite}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y)))$

4.11.4 Clamping

Signature

$\text{Clamp}_{f_x, f_{lo}, f_{hi}, f_r, \rho}(x, lo, hi) \rightarrow r$

Parameters

f_x : format of operand x
 f_{lo} : format of lower bound lo
 f_{hi} : format of upper bound hi
 f_r : format of result r
 ρ : projection specification

Operands

x : floating-point value, format f_x
 lo : floating-point value, format f_{lo}
 hi : floating-point value, format f_{hi}

Result

r : floating-point value, format f_r

Behavior

$\omega\text{Clamp}(\text{NaN}, *, *) \rightarrow \text{NaN}$
 $\omega\text{Clamp}(*, \text{NaN}, *) \rightarrow \text{NaN}$
 $\omega\text{Clamp}(*, *, \text{NaN}) \rightarrow \text{NaN}$
 $\omega\text{Clamp}(*, Lo, Hi) \text{ if } Lo > Hi \rightarrow \text{NaN}$
 $\omega\text{Clamp}(*, +\infty, +\infty) \rightarrow +\infty$
 $\omega\text{Clamp}(*, -\infty, -\infty) \rightarrow -\infty$
 $\omega\text{Clamp}(*, *, -\infty) \rightarrow \text{NaN}$
 $\omega\text{Clamp}(*, +\infty, *) \rightarrow \text{NaN}$
 $\omega\text{Clamp}(+\infty, *, +\infty) \rightarrow +\infty$
 $\omega\text{Clamp}(+\infty, *, Hi) \rightarrow Hi$
 $\omega\text{Clamp}(-\infty, -\infty, *) \rightarrow -\infty$
 $\omega\text{Clamp}(-\infty, Lo, *) \rightarrow Lo$
 $\omega\text{Clamp}(X, -\infty, +\infty) \rightarrow X$
 $\omega\text{Clamp}(X, -\infty, Hi) \text{ if } X \leq Hi \rightarrow X$
 $\omega\text{Clamp}(X, -\infty, Hi) \text{ if } X > Hi \rightarrow Hi$
 $\omega\text{Clamp}(X, Lo, +\infty) \text{ if } X \leq Lo \rightarrow Lo$
 $\omega\text{Clamp}(X, Lo, +\infty) \text{ if } X > Lo \rightarrow X$
 $\omega\text{Clamp}(X, Lo, Hi) \rightarrow \text{if } X \leq Lo \text{ then } Lo \text{ else if } X \geq Hi \text{ then } Hi \text{ else } X$
 $\text{Clamp}(x, lo, hi) \rightarrow \omega\text{Project}_{f_r, \rho}(\omega\text{Clamp}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_{lo}}(lo), \omega\text{Decode}_{f_{hi}}(hi)))$

4.12 Comparisons

Comparison operators take two operands and return a Boolean value.

Signature

$\text{Compare}_{Op_{f_x, f_y}}(x, y) \rightarrow b$

Parameters

f_x : format of operand x

f_y : format of operand y

Operands

x : floating-point value, format f_x

y : floating-point value, format f_y

Result

b : boolean value

Behavior

$\omega\text{CompareLess}(\text{NaN}, *) \rightarrow \text{False}$

$\omega\text{CompareLess}(*, \text{NaN}) \rightarrow \text{False}$

$\omega\text{CompareLess}(+\infty, *) \rightarrow \text{False}$

$\omega\text{CompareLess}(*, +\infty) \rightarrow \text{True}$

$\omega\text{CompareLess}(-\infty, -\infty) \rightarrow \text{False}$

$\omega\text{CompareLess}(-\infty, *) \rightarrow \text{True}$

$\omega\text{CompareLess}(*, -\infty) \rightarrow \text{False}$

$\omega\text{CompareLess}(X, Y) \rightarrow X < Y$

$\text{CompareLess}(x, y) \rightarrow \omega\text{CompareLess}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y))$

$\omega\text{CompareLessEqual}(\text{NaN}, *) \rightarrow \text{False}$

$\omega\text{CompareLessEqual}(*, \text{NaN}) \rightarrow \text{False}$

$\omega\text{CompareLessEqual}(*, +\infty) \rightarrow \text{True}$

$\omega\text{CompareLessEqual}(-\infty, *) \rightarrow \text{True}$

$\omega\text{CompareLessEqual}(+\infty, *) \rightarrow \text{False}$

$\omega\text{CompareLessEqual}(*, -\infty) \rightarrow \text{False}$

$\omega\text{CompareLessEqual}(X, Y) \rightarrow X \leq Y$

$\text{CompareLessEqual}(x, y) \rightarrow \omega\text{CompareLessEqual}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y))$

$\omega\text{CompareEqual}(\text{NaN}, *) \rightarrow \text{False}$

$\omega\text{CompareEqual}(*, \text{NaN}) \rightarrow \text{False}$

$\omega\text{CompareEqual}(+\infty, +\infty) \rightarrow \text{True}$

$\omega\text{CompareEqual}(-\infty, -\infty) \rightarrow \text{True}$

$\omega\text{CompareEqual}(+\infty, *) \rightarrow \text{False}$

$\omega\text{CompareEqual}(-\infty, *) \rightarrow \text{False}$

$\omega\text{CompareEqual}(*, -\infty) \rightarrow \text{False}$

$\omega\text{CompareEqual}(*, +\infty) \rightarrow \text{False}$

$\omega\text{CompareEqual}(X, Y) \rightarrow X = Y$

$\text{CompareEqual}(x, y) \rightarrow \omega\text{CompareEqual}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y))$

[Comparisons continued on next page]

$\omega\text{CompareGreaterEqual}(\text{NaN}, *) \rightarrow \text{False}$
 $\omega\text{CompareGreaterEqual}(*, \text{NaN}) \rightarrow \text{False}$
 $\omega\text{CompareGreaterEqual}(+\infty, *) \rightarrow \text{True}$
 $\omega\text{CompareGreaterEqual}(-\infty, -\infty) \rightarrow \text{True}$
 $\omega\text{CompareGreaterEqual}(-\infty, *) \rightarrow \text{False}$
 $\omega\text{CompareGreaterEqual}(*, +\infty) \rightarrow \text{False}$
 $\omega\text{CompareGreaterEqual}(*, -\infty) \rightarrow \text{True}$
 $\omega\text{CompareGreaterEqual}(X, Y) \rightarrow X \geq Y$
 $\text{CompareGreaterEqual}(x, y) \rightarrow \omega\text{CompareGreaterEqual}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y))$
 $\omega\text{CompareGreater}(\text{NaN}, *) \rightarrow \text{False}$
 $\omega\text{CompareGreater}(*, \text{NaN}) \rightarrow \text{False}$
 $\omega\text{CompareGreater}(*, +\infty) \rightarrow \text{False}$
 $\omega\text{CompareGreater}(-\infty, -\infty) \rightarrow \text{False}$
 $\omega\text{CompareGreater}(*, -\infty) \rightarrow \text{True}$
 $\omega\text{CompareGreater}(+\infty, *) \rightarrow \text{True}$
 $\omega\text{CompareGreater}(-\infty, *) \rightarrow \text{False}$
 $\omega\text{CompareGreater}(X, Y) \rightarrow X > Y$
 $\text{CompareGreater}(x, y) \rightarrow \omega\text{CompareGreater}(\omega\text{Decode}_{f_x}(x), \omega\text{Decode}_{f_y}(y))$

4.12.1 Total order predicate

Signature

$\text{TotalOrder}_{f_x, f_y}(x, y) \rightarrow b$

Parameters

f_x : format of operand x
 f_y : format of operand y

Operands

x : floating-point value, format f_x
 y : floating-point value, format f_y

Result

b : Boolean value

Behavior

$\text{TotalOrder}(\text{NaN}, x) \rightarrow \text{True}$
 $\text{TotalOrder}(x, \text{NaN}) \rightarrow \text{False}$
 $\text{TotalOrder}(x, y) \rightarrow \text{CompareLessEqual}_{f_x, f_y}(x, y)$

NOTE 1—The $\text{TotalOrder}(x, y)$ predicate provides a total ordering over each format's value set.

NOTE 2— TotalOrder is not defined when either format is an external format.

4.12.2 Comparison operator symbols

This section defines the mapping between comparison operator symbols that a system may make available with floating-point values as operands.

Table 1—Comparison predicates and negations

Mathematical symbol	Predicate
$x = y$	<code>CompareEqual(x, y)</code>
$x > y$	<code>CompareGreater(x, y)</code>
$x \geq y, x \geqslant y$	<code>CompareGreaterEqual(x, y)</code>
$x < y$	<code>CompareLess(x, y)</code>
$x \leq y, x \leqslant y$	<code>CompareLessEqual(x, y)</code>
$x \neq y, x \neqslant y$	<code>not CompareEqual(x, y)</code>

4.13 Predicates and classification

The classification operations comprise: 1) a set of predicate functions with a Boolean return value, taking a single operand; 2) a classifier operation $\text{Class}(x)$ that returns a single value of enumeration type, describing the operand's properties.

Signature

$\text{IsClass}_f(x) \rightarrow b$

Parameters

f : format of operand x

Operands

x : floating-point value, in format f

Result

b : boolean value

Behavior

$\omega\text{IsZero}(\text{NaN}) \rightarrow \text{False}$

$\omega\text{IsZero}(\pm\infty) \rightarrow \text{False}$

$\omega\text{IsZero}(X) \rightarrow X = 0$

$\text{IsZero}(x) \rightarrow \omega\text{IsZero}(\omega\text{Decode}_f(x))$

$\omega\text{IsOne}(\text{NaN}) \rightarrow \text{False}$

$\omega\text{IsOne}(\pm\infty) \rightarrow \text{False}$

$\omega\text{IsOne}(X) \rightarrow X = 1$

$\text{IsOne}(x) \rightarrow \omega\text{IsOne}(\omega\text{Decode}_f(x))$

$\text{IsNaN}(x) \rightarrow \omega\text{Decode}_f(x) \in \{\text{NaN}\}$

$\text{IsInfinite}(x) \rightarrow \omega\text{Decode}_f(x) \in \{-\infty, +\infty\}$

$\text{IsFinite}(x) \rightarrow \text{not IsInfinite}(x) \text{ and not IsNaN}(x)$

$\omega\text{IsSignMinus}(\text{NaN}) \rightarrow \text{False}$

$\omega\text{IsSignMinus}(+\infty) \rightarrow \text{False}$

$\omega\text{IsSignMinus}(-\infty) \rightarrow \text{True}$

$\omega\text{IsSignMinus}(X) \rightarrow X < 0$

$\text{IsSignMinus}(x) \rightarrow \omega\text{IsSignMinus}(\omega\text{Decode}_f(x))$

$\text{IsNormal}(x) \text{ if } \text{IsZero}(x) \text{ or } \text{IsInfinite}(x) \text{ or } \text{IsNaN}(x) \rightarrow \text{False}$

$\text{IsNormal}(x) \rightarrow |\omega\text{Decode}_f(x)| \geq \omega\text{Decode}_f(\text{MinNormalOf}(f))$

$\text{IsSubnormal}(x) \text{ if } \text{IsZero}(x) \text{ or } \text{IsInfinite}(x) \text{ or } \text{IsNaN}(x) \rightarrow \text{False}$

$\text{IsSubnormal}(x) \rightarrow \text{not IsNormal}(x)$

4.13.1 Classifier operation

The classifier operation $\text{Class}(x)$ tells which of the eight classes x falls into as defined by Table 2.

Signature

$\text{Class}_f(x) \rightarrow c$

Parameters

f : format of operand x

Operands

x : floating-point value, format f

Result

c : enumeration

Behavior

$\text{Class}(x) \rightarrow \text{ClassEnum}$

Table 2—Classifier operation

ClassEnum	Condition
ClsNaN	IsNaN(x)
ClsNegativeInfinity	IsInfinite(x) and IsSignMinus(x)
ClsNegativeNormal	IsNormal(x) and IsSignMinus(x)
ClsNegativeSubnormal	IsSubnormal(x) and IsSignMinus(x)
ClsZero	IsZero(x)
ClsPositiveSubnormal	IsSubnormal(x) and not IsSignMinus(x)
ClsPositiveNormal	IsNormal(x) and not IsSignMinus(x)
ClsPositiveInfinity	IsInfinite(x) and not IsSignMinus(x)

4.14 Format-level operations

Certain operations act on formats rather than values, for example, determining the precision of a format.¹³

The following operations return integers or enumerations:

Operation	Format					
	Binary{K, P, Σ , Δ }		binary64	binary32	binary16	BFloat16
	$\Sigma = \text{Signed}$	$\Sigma = \text{Unsigned}$				
BitwidthOf(f)	K	K	64	32	16	16
PrecisionOf(f)	P	P	53	24	11	8
SignednessOf(f)	Signed	Unsigned	Signed	Signed	Signed	Signed
DomainOf(f)	Δ	Δ	Extended	Extended	Extended	Extended
ExponentBitwidthOf(f)	$K - P$	$K - P + 1$	11	8	5	8
TrailingSignificandBitwidthOf(f)	$P - 1$	$P - 1$	52	23	10	7
ExponentBiasOf(f)	2^{K-P-1}	2^{K-P}	1023	127	15	127

The following operations return floating-point values, in the format f :

Operation	Description
MaxFiniteOf(f)	Maximum finite value representable in format f .
MinFiniteOf(f)	Minimum finite value representable in format f . In signed formats, $\omega_{\text{Decode}_f}(\text{MinFiniteOf}(f)) = -\omega_{\text{Decode}_f}(\text{MaxFiniteOf}(f))$. In unsigned formats, $\text{MinFiniteOf}(f) = 0_f$.
MinPositiveOf(f)	Minimum strictly positive value representable in format f .
MaxSubnormalOf(f)	Maximum positive subnormal value representable in format f , or NaN if no values are subnormal.
MinNormalOf(f)	Minimum positive normal value representable in format f .

4.15 Projection specification operations

These operations query projection specifications and return enumerations:

RoundOf(ρ)	Rounding mode of projection specification ρ , such that $\text{RoundOf}((r, s)) = r$.
SatOf(ρ)	Saturation mode of projection specification ρ , such that $\text{SatOf}((r, s)) = s$.

¹³The external formats binary64, binary32, binary16 are from [7], and BFloat16 from [8].

4.16 Next greater than and next less than

Return the least (or greatest) floating-point value that compares greater than (or less than) the argument. If no such value exists, return NaN.

Signature

$\text{NextGreaterThan}_f(x) \rightarrow r$
 $\text{NextLessThan}_f(x) \rightarrow r$

Parameters

f : format of operand x and result r

Operands

x : floating-point value, format f

Result

r : floating-point value, format f

Behavior

$\text{NextGreaterThanAux}(*, *, x) \text{ if } \text{IsNaN}(x) \rightarrow x$
 $\text{NextGreaterThanAux}(\text{Signed}, *, 2^{K-1}) \rightarrow 1$
 $\text{NextGreaterThanAux}(*, \text{Extended}, \text{Inf}) \rightarrow \omega\text{Encode}_f(\text{NaN})$
 $\text{NextGreaterThanAux}(*, \text{Finite}, \text{MaxFinite}) \rightarrow \omega\text{Encode}_f(\text{NaN})$
 $\text{NextGreaterThanAux}(*, \text{Extended}, \text{MaxFinite}) \rightarrow \text{Inf}$
 $\text{NextGreaterThanAux}(\text{Signed}, \text{Extended}, -\text{Inf}) \rightarrow \text{MinFinite}$
 $\text{NextGreaterThanAux}(\text{Signed}, *, \text{SmallestNegative}) \rightarrow 0$
 $\text{NextGreaterThanAux}(\text{Signed}, *, x) \text{ if } \text{IsSignMinus}(x) \rightarrow x - 1$
 $\text{NextGreaterThanAux}(*, *, x) \rightarrow x + 1$ (See §4.1 for integer arithmetic on code points.)

$\text{NextGreaterThan}(x) \rightarrow \text{NextGreaterThanAux}(\text{SignednessOf}(f), \text{DomainOf}(f), x)$

$\text{NextLessThanAux}(*, *, x) \text{ if } \text{IsNaN}(x) \rightarrow x$
 $\text{NextLessThanAux}(\text{Signed}, *, 2^{K-1}) \rightarrow 2^{K-1} + 1$
 $\text{NextLessThanAux}(*, \text{Finite}, \text{MinFinite}) \rightarrow \omega\text{Encode}_f(\text{NaN})$
 $\text{NextLessThanAux}(\text{Unsigned}, *, 0) \rightarrow \omega\text{Encode}_f(\text{NaN})$
 $\text{NextLessThanAux}(\text{Signed}, \text{Extended}, -\text{Inf}) \rightarrow \omega\text{Encode}_f(\text{NaN})$
 $\text{NextLessThanAux}(\text{Signed}, \text{Extended}, \text{MinFinite}) \rightarrow -\text{Inf}$
 $\text{NextLessThanAux}(*, \text{Extended}, \text{Inf}) \rightarrow \text{MaxFinite}$
 $\text{NextLessThanAux}(\text{Signed}, *, 0) \rightarrow \text{SmallestNegative}$
 $\text{NextLessThanAux}(\text{Signed}, *, x) \text{ if } \text{IsSignMinus}(x) \rightarrow x + 1$
 $\text{NextLessThanAux}(*, *, x) \rightarrow x - 1$

$\text{NextLessThan}(x) \rightarrow \text{NextLessThanAux}(\text{SignednessOf}(f), \text{DomainOf}(f), x)$

where

$K = \text{BitwidthOf}(f)$
 $\text{MaxFinite} = \text{MaxFiniteOf}(f)$
 $\text{MinFinite} = \text{MinFiniteOf}(f)$
 $\text{SmallestNegative} = \omega\text{Encode}_f(-\omega\text{Decode}_f(\text{MinPositiveOf}(f)))$ (The negative number of least magnitude)

NOTE 1—*SmallestNegative* is not expanded for unsigned formats.

NOTE 2—These operations follow IEEE-754 nextUp and nextDown operations, with extensions for signedness and

domain. The wording, using `nextUp` as an example, is extended from “least floating-point number that compares greater than” to “least floating-point number that compares greater than, or NaN”. This requires that $\text{Inf} \rightarrow \text{NaN}$, while IEEE-754 defines $\text{nextUp}(+\infty) = +\infty$, so new names are chosen in this document.

5 Block operations

5.1 General

A block is a pair $(s, [x_1, \dots, x_B])$ comprising a scale factor s in format f_s and a sequence of one or more elements x_i , each in format f_x .

NOTE— Scaled operations are available as block operations using blocks of one element (§5.8).

5.2 Definitions

Some operations in this section make use of the function `reduce`, defined as follows.

For binary function f , and arguments $x_1, \dots, x_m$:

$$\begin{aligned} \text{reduce}(f, [x_1, x_2]) &= f(x_1, x_2) \\ \text{reduce}(f, [x_1, \dots, x_m]) &= \text{reduce}(f, [f(x_1, x_2), x_3, \dots, x_m]) \quad \text{for } m \geq 3 \end{aligned}$$

5.3 Stochastic rounding in block operations

When stochastic rounding is applied to operations returning sequences, the meaning of the random bits R is altered to mean a sequence of integers $R = [R_1, \dots, R_B]$. Specifically, when $\text{RoundOf}(\rho) = \text{Stochastic}[A, B, C]_{N,R}$, define $\rho[i] := (\text{Stochastic}[A, B, C]_{N,R_i}, \text{SatOf}(\rho))$. For such operations, the constraint that R is an integer such that $0 \leq R < 2^N$ is replaced by a constraint that R_i is an integer and $0 \leq R_i < 2^N$. For non-stochastic rounding modes, define $\rho[i] := \rho$.

NOTE—While the specification is in terms of a sequence of integers, an implementation may choose to define that sequence in any appropriate manner, for example by indexing into an array of integers, or by sampling from a random number generator specified by R , or by the calling of a defined function that generates the sequence.

5.4 Internal functions

5.4.1 Decoding

Signature

$$\omega\text{BlockDecode}_{B,f_s,f_x}(s, [x_1, \dots, x_B]) \rightarrow [Z_1, \dots, Z_B]$$

Parameters

B : block size

f_s : format of scale factor s

f_x : format of elements x

Operands

s : scale factor, in format f_s

$\mathbf{x}$: sequence of floating-point values in format f_x

Result

$\mathbf{Z}$: sequence of closed extended real values

Behavior

$$\omega\text{BlockDecode}(s, [x_1, \dots, x_B]) \rightarrow [Z_1, \dots, Z_B]$$

where

$$Z_i = \omega\text{Multiply}(\omega\text{Decode}_{f_s}(s), \omega\text{Decode}_{f_x}(x_i))$$

5.4.2 Projection

Convert a sequence of closed extended reals $X_{1..B}$ to a block $(s, [r_1, \dots, r_B])$, with scale factor s supplied as an operand.

Signature

$\omega\text{BlockProject}_{B, f_s, f_r, \rho}(s, [X_1, \dots, X_B]) \rightarrow [r_1, \dots, r_B]$

Parameters

B : block size
 f_s : format of result scale factor s
 f_r : format of result elements r
 ρ : projection specification for elements

Operands

s : result scale factor in format f_s
 $\mathbf{X}$: sequence of closed extended reals

Result

$\mathbf{r}$: sequence of floating-point values in format f_r

Behavior

$\omega\text{BlockProject}(s, [X_1, \dots, X_B]) \rightarrow [r_1, \dots, r_B]$

where

$$\begin{aligned} S &= \omega\text{Decode}_{f_s}(s) \\ Z_i &= \begin{cases} \text{NaN} & \text{if } S \text{ is NaN or } X_i \text{ is NaN} \\ 0 & \text{if } S = 0 \\ \text{sgn}(X_i) \times \text{sgn}(S) & \text{if } S = \pm\infty \\ \omega\text{Divide}(X_i, S) & \text{otherwise} \end{cases} \\ r_i &= \omega\text{Project}_{f_r, \rho[i]}(Z_i) \end{aligned}$$

NOTE 1—If the scale factor s is zero, all non-NaN result elements are zero.

NOTE 2—If the scale factor s is infinite, all nonzero, non-NaN result elements are ± 1 .

5.5 Conversion of blocks

5.5.1 Conversion from a block

Convert a block $(s, [x_1, \dots, x_B])$ to a sequence of floating-point values $r_{1..B}$.

Signature

$\text{ConvertFromBlock}_{B, f_s, f_x, f_r, \rho}(s, [x_1, \dots, x_B]) \rightarrow [r_1, \dots, r_B]$

Parameters

B : block size
 f_s : format of scale factor operand s
 f_x : format of operand elements x_i
 f_r : format of result elements r
 ρ : projection specification for result elements

Operands

s : scale factor in format f_s
 $\mathbf{x}$: sequence of floating-point values in format f_x

Result

$\mathbf{r}$: sequence of floating-point values in format f_r

Behavior

$\text{ConvertFromBlock}(s, [x_1, \dots, x_B]) \rightarrow [r_1, \dots, r_B]$
where
 $[Z_1, \dots, Z_B] = \omega\text{BlockDecode}(s, [x_1, \dots, x_B])$
 $r_i = \omega\text{Project}_{f_r, \rho[i]}(Z_i)$

5.5.2 Conversion to a block

Convert a sequence of floating-point values $x_{1..B}$ to a block $(s, [r_1, \dots, r_B])$, with scale factor supplied as an operand.

Signature

$\text{ConvertToBlock}_{B, f_x, f_s, f_r, \rho}([x_1, \dots, x_B], s) \rightarrow (s, [r_1, \dots, r_B])$

Parameters

B : block size
 f_x : format of operand elements x_i
 f_s : format of result scale factor s
 f_r : format of result elements r
 ρ : projection specification for result elements

Operands

$\mathbf{x}$: sequence of floating-point values in format f_x
 s : result scale factor, a floating-point value in format f_s

Result

s : result scale factor, a floating-point value in format f_s
 $\mathbf{r}$: sequence of floating-point values in format f_r

Behavior

$\text{ConvertToBlock}([x_1, \dots, x_B], s) \rightarrow (s, [r_1, \dots, r_B])$

where

$X_i = \omega\text{Decode}_{f_x}(x_i)$

$[r_1, \dots, r_B] = \omega\text{BlockProject}_{B, f_s, f_r, \rho}(s, [X_1, \dots, X_B])$

5.5.3 Conversion to a block with scale factor computation

Convert a sequence of floating-point values $x_{1..B}$ to a block $(s, [r_1, \dots, r_B])$, computing the scale factor as a maximum over finite absolute values of x_i .

Signature

$\text{ConvertToBlockMaxAbsFinite}_{B, f_x, f_s, f_r, \rho_s, \rho}([x_1, \dots, x_B]) \rightarrow (s, [r_1, \dots, r_B])$

Parameters

B : block size
 f_x : format of operand elements x_i
 f_s : format of result scale factor s
 f_r : format of result elements r
 ρ_s : projection specification for result scale factor
 ρ : projection specification for result elements

Operands

$\mathbf{x}$: sequence of floating-point values in format f_x

Result

s : result scale factor, a floating-point value in format f_s
 $\mathbf{r}$: sequence of floating-point values in format f_r

Behavior

$\text{ConvertToBlockMaxAbsFinite}([x_1, \dots, x_B]) \rightarrow (s, [r_1, \dots, r_B])$

where

$X_i = \omega\text{Decode}_{f_x}(x_i)$
 $M_i = \omega\text{Abs}(X_i)$
 $S = \text{reduce}(\omega\text{MaximumFinite}, [\text{NaN}, M_1, \dots, M_B])$
 $s = \omega\text{Project}_{f_s, \rho_s}(S)$
 $[r_1, \dots, r_B] = \omega\text{BlockProject}_{B, f_s, f_r, \rho}(s, [X_1, \dots, X_B])$

NOTE 1—If all x_i are NaN, the result scale factor and elements will be NaN.

NOTE 2—If all x_i are infinite, and f_s is in the extended domain, the result scale factor will be Inf or MaxFiniteOf(f_s) and the elements will be ± 1 or $\pm\text{MaxFiniteOf}(f_r)$.

NOTE 3—If some x_i are infinite, they do not influence the scale of finite elements.

NOTE 4—If the maximum value S rounds to zero under ρ_s , the result scale factor will be zero, and all non-NaN result elements will be zero.

NOTE 5—The projection specifications for the scale factor and elements allow implementation of some common strategies, e.g., rounding s using TowardPositive and saturating r .

5.6 Block reduction operations

5.6.1 Sum and product

Signature

BlockReduceAdd_{*B, f_s, f_x, f_r, ρ*}((*s*, [*x*₁, ..., *x*_{*B*}])) → *r*
BlockReduceMultiply_{*B, f_s, f_x, f_r, ρ*}((*s*, [*x*₁, ..., *x*_{*B*}])) → *r*

Parameters

B : block size
f_s : format of scale factor operand *s*
f_x : format of operand elements *x_i*
f_r : format of result *r*
ρ : projection specification

Operands

s : scale factor, a floating-point value in format *f_s*
x : sequence of floating-point values in format *f_x*

Result

r : floating-point value in format *f_r*

Behavior

BlockReduceAdd((*s*, [*x*₁, ..., *x*_{*B*}])) → *r*
 where
 [*X*₁, ..., *X*_{*B*}] = ωBlockDecode_{*B, f_s, f_x*}(*s*, [*x*₁, ..., *x*_{*B*}]))
 R = reduce(ωAdd, [0, *X*₁, ..., *X*_{*B*}]))
 r = ωProject_{*f_r, ρ*}(*R*)

BlockReduceMultiply((*s*, [*x*₁, ..., *x*_{*B*}])) → *r*
 where
 [*X*₁, ..., *X*_{*B*}] = ωBlockDecode_{*B, f_s, f_x*}(*s*, [*x*₁, ..., *x*_{*B*}]))
 R = reduce(ωMultiply, [1, *X*₁, ..., *X*_{*B*}]))
 r = ωProject_{*f_r, ρ*}(*R*)

5.6.2 Dot product

Dot product of two blocks of floating-point values.

Signature

$\text{BlockDotProduct}_{B, f_{sx}, f_x, f_{sy}, f_y, f_r, \rho}((s_x, [x_1, \dots, x_B]), (s_y, [y_1, \dots, y_B])) \rightarrow r$

Parameters

B : block size
 f_{sx} : format of scale factor operand s_x
 f_x : format of operand elements x_i
 f_{sy} : format of scale factor operand s_y
 f_y : format of operand elements y_i
 f_r : format of result r
 ρ : projection specification

Operands

s_x : scale factor for x , a floating-point value in format f_{sx}
 $\mathbf{x}$: sequence of floating-point values in format f_x
 s_y : scale factor for y , a floating-point value in format f_{sy}
 $\mathbf{y}$: sequence of floating-point values in format f_y

Result

r : floating-point value in format f_r

Behavior

$\text{BlockDotProduct}((s_x, [x_1, \dots, x_B]), (s_y, [y_1, \dots, y_B])) \rightarrow r$
where
 $[X_1, \dots, X_B] = \omega\text{BlockDecode}_{B, f_{sx}, f_x}(s_x, [x_1, \dots, x_B])$
 $[Y_1, \dots, Y_B] = \omega\text{BlockDecode}_{B, f_{sy}, f_y}(s_y, [y_1, \dots, y_B])$
 $P_i = \omega\text{Multiply}(X_i, Y_i)$
 $R = \text{reduce}(\omega\text{Add}, [0, P_1, \dots, P_B])$
 $r = \omega\text{Project}_{f_r, \rho}(R)$

5.7 Elementwise operations on blocks

Operations on blocks are declared according to the following schema, where BlockOp is defined in terms of ωOp .

Note that the scale factor of the result block s_r is supplied as an *input* operand. Implementations are free to supply any method of computation of this scale factor, and to supply an operation in which that computation is composed with the elementwise operation.

Valid substitutions for Op are, with arity n :

- Unary ($n = 1$): Convert, Abs, Negate, Sqrt, RSqrt, Recip, Exp, Log, ExpMinusOne, LogOnePlus, Exp2, Log2, Sin, Cos, Tan, ArcSin, ArcCos, ArcTan, Sinh, Cosh, Tanh, ArcSinh, ArcCosh, ArcTanh, SinPi, CosPi, TanPi, ArcSinPi, ArcCosPi, ArcTanPi, Softplus;
- Binary ($n = 2$): CopySign, Add, Subtract, Multiply, Divide, Hypot, ArcTan2, ArcTan2Pi, Maximum, Minimum, MaximumNumber, MinimumNumber, MaximumMagnitude, MinimumMagnitude, MaximumMagnitudeNumber, MinimumMagnitudeNumber, MinimumFinite, MaximumFinite;
- Ternary ($n = 3$): FMA, FAA, Clamp.

Signature

$$\text{BlockOp}_{B, (f_{s1}, f_{x1}), \dots, (f_{sn}, f_{xn}), f_s, f_r, \rho}((s_1, [x_{11}, \dots, x_{1B}]), \dots, (s_n, [x_{n1}, \dots, x_{nB}]), s_r) \rightarrow (s_r, [r_1, \dots, r_B])$$

Parameters

B : block size
 f_{s1} : format of first operand scale factor s_1
 f_{x1} : format of first operand element x_{1i}
 $\dots$: ...
 f_{sn} : format of n^{th} operand scale factor s_n
 f_{xn} : format of n^{th} operand elements x_{ni}
 f_s : format of scale factor of result s_r
 f_r : format of result elements r
 ρ : projection specification for result elements

Operands

s_1 : scale factor, a floating-point value in format f_{s1}
 $\mathbf{x}_1$: sequence of floating-point values in format f_{x1}
 $\dots$: ...
 s_n : scale factor, a floating-point value in format f_{sn}
 $\mathbf{x}_n$: sequence of floating-point values in format f_{xn}
 s_r : result scale factor, a floating-point value in format f_s

Result

s_r : result scale factor, copied from operand
 $\mathbf{r}$: sequence of floating-point values in format f_r

Behavior

$$\text{BlockOp}((s_1, [x_{11}, \dots, x_{1B}]), \dots, (s_n, [x_{n1}, \dots, x_{nB}]), s_r) \rightarrow (s_r, [r_1, \dots, r_B])$$

where

$$\begin{aligned} [X_{11}, \dots, X_{1B}] &= \omega \text{BlockDecode}_{B, f_{s1}, f_{x1}}(s_1, [x_{11}, \dots, x_{1B}]) \\ &\dots \\ [X_{n1}, \dots, X_{nB}] &= \omega \text{BlockDecode}_{B, f_{sn}, f_{xn}}(s_n, [x_{n1}, \dots, x_{nB}]) \\ Z_i &= \omega Op(X_{1i}, \dots, X_{ni}) \quad \forall i \in \{1, \dots, B\} \\ [r_1, \dots, r_B] &= \omega \text{BlockProject}_{B, f_s, f_r, \rho}(s_r, [Z_1, \dots, Z_B]) \end{aligned}$$

5.8 Scaled operations via block operations

A block size of $B = 1$ provides scaled operations.

An n -ary scaled operation is defined as follows:

$$\text{Scaled } Op_{(f_{s1}, f_{x1}), \dots, (f_{sn}, f_{xn}), f_r, \rho}(s_1, x_1, \dots, s_n, x_n) \rightarrow r$$

where

$$(1, [r]) = \text{Block } Op_{1, (f_{s1}, f_{x1}), \dots, (f_{sn}, f_{xn}), f_s, f_r, \rho'}((s_1, [x_1]), \dots, (s_n, [x_n]), 1)$$

where the result scale factor format f_s is any format which allows the exact representation of the number 1. This applies to all conforming formats in this document.

The notation ρ' denotes the projection specification

$$\rho' := \begin{cases} (\text{Stochastic}[A, B, C]_{N, [R]}, \text{SatOf}(\rho)) & \text{if } \text{RoundOf}(\rho) = \text{Stochastic}[A, B, C]_{N, R} \\ \rho & \text{otherwise} \end{cases}$$

where $[R]$ is the sequence of length one whose sole element is R .

NOTE—A choice of scale factor format with $P = 1$ allows scale factors to be restricted to powers of two.

Appendices

Annex A

(informative)

Rationales and discussion

A.1 General

The principle underpinning the design of floating-point formats is suitability for use in machine learning systems, where narrow bitwidths are important for reducing energy usage, conserving memory space, and delivering timely results. Therefore, rationales consider the cost of a feature in terms of number of code points required and its utility in machine learning systems.

A.2 Not a number (NaN)

Datum sets include exactly one NaN.

NaN is obtained from operation results that are outside the set of numerical values, e.g., division of zero by zero, or addition of positive and negative infinities. Other floating-point systems define multiple NaN values. These encodings are used to allow different exceptional conditions to be distinguished.

In the context of machine learning systems, uses of NaN include:

- Debugging of code running on accelerator hardware. In machine learning accelerators, exceptions may be difficult or expensive to convey back to user code, so it is common practice to allow NaN values to propagate through calculations to indicate that an error has occurred.
- Use as a sentinel value. In some datasets, for example, where individual element values may be missing or out of range, a sentinel may be used to record the position of these values. In many cases, this requires less memory than storing such information out-of-band, such as in a coordinate-list (COO) format array. In some cases, $\pm\text{Inf}$ can be used as a missing value, but given the restricted range of the formats, it is likely that infinity will be used as a separate indicator of rounding from values outside of the finite range.
- The use of multiple NaN payloads is known in statistical code (e.g., NaN and “not available” or NA), but it is not widely used in large-scale machine learning systems.

For our purposes, supporting multiple NaNs would reduce the already limited encoding space (e.g., occupying code points where the exponent field is all ones, reducing dynamic range) and potentially increase hardware complexity.

A.3 Zero

Datum sets include exactly one zero.

The inclusion of negative zero (-0) would incur the cost of an additional code point. Given the decision to encode only a single NaN, placing that NaN at the code point where negative zero would be encoded enables the strictly positive and strictly negative number ranges to be symmetric for signed formats.

A key rationale for including -0 in IEEE-754 was the consistent implementation of branch cuts in the ArcTan2 function and the complex trigonometric functions [9, 10]. The ArcTan2 function is rarely used in deep learning applications. The related ArcTan function is common in deep learning, however it is generally used as an activation function, rather than a trigonometric operation.

A secondary reason for providing -0 is the hardware simplification offered by its presence in the implementation of sign/magnitude arithmetic. However, current practice has made it evident that the small hardware simplification has not been sufficient to balance the loss of one code point.

Another reason for including -0 in IEEE-754 was to allow the reciprocal of a signed infinity to be the corresponding signed zero. In this standard the reciprocal of either infinity is zero, and a corollary of that decision is that dividing by zero produces NaN, not an infinity.

The use of integer comparisons in sorting might argue against placing NaN at the negative zero code point. For example, the JAX machine learning framework is known to sort using integer comparison [5]. However, such sorting still requires $O(n)$ additional processing steps to enable the use of two's-complement integer comparison, and already has special treatment of NaN and -0 . Eliminating -0 and placing NaN in the -0 position imposes negligible additional burden.

A.4 Infinities

Representing infinite values requires two code points in a signed format, and for narrow formats, the reduction in number of finite values may be significant. Hence formats are defined using the finite and extended domains.

Infinite values are used widely in machine learning systems. Examples of such usage are:

- Mask values, for example in transformer models in machine learning [11].
- Representation of overflow, for example to adjust dynamic loss scaling factors [13].

The following example shows that in certain machine learning computations, it is insufficient to substitute an infinity with the largest finite value.

[Continued on next page]

Example:

Consider the use of infinity in computation of attention masks. These values, assembled in a mask vector M with values $M_i \in \{0, \infty\}$, are typically combined with computed values A in a computation such as:

$$\log \left(\sum_i \exp(\tau \times (A_i - M_i)) \right)$$

where $\tau > 0$ is a “temperature” or “base” parameter [4]. This calculation depends on the fact that $\exp(\tau \times (A_i - \infty)) = 0$. The operation sequence $\log(\sum_i \exp(v_i))$ is typically fused into a single block operation $\text{logsumexp}(v)$.

For formats without infinities, $M_i = \infty$ is replaced by a large float (e.g., the largest finite Binary8p4sf value is 240.0). This is not in itself a difficulty: if all the A values are bounded (e.g., the results of a softmax operation are bounded above by 1.0), then $\exp(1.0 - 240.0)$ is an extremely small number, sufficiently small to round to zero. Therefore, an explicit representation of infinity is *not* needed in order for this computation to yield its desired value.

However, careful implementations do not execute the calculation as written; instead, the implementation of logsumexp makes use of the transformation

$$\text{logsumexp}(v) \rightarrow \text{logsumexp}(v - \max(v)) + \max(v)$$

Without the “sticky” properties of Inf, this would produce incorrect answers, as follows.

For example, compare a format using the finite domain where $\text{maxFinite}=240$, and a format using the extended domain. In both cases, the value 224.0 is representable, and the logsumexp calculation on a two-element vector might be, in a format with infinity:

$$\text{logsumexp}(\tau \times [-224.0, -\infty]) \rightarrow \text{logsumexp}(\tau \times [0, -\infty]) + \tau \times (-224.0),$$

and in a format without infinity:

$$\text{logsumexp}(\tau \times [-224.0, -240.0]) \rightarrow \text{logsumexp}(\tau \times [0, -16.0]) + \tau \times (-224.0).$$

If $\tau = 1$ and all calculations are done in 8-bit floating-point, then the two answers are the same, because $\exp(-16) \approx 1.1 \times 10^{-7}$, which will round to zero in all precisions $P > 2$. However, if τ is small, or calculations are done in mixed precision, the loss of “stickiness” will silently yield unexpected answers. It is not expected that the full calculation would be done in 8-bit floating-point, but the subtraction of the maximum value (and computation of the maximum) might reasonably be done in 8-bit floating-point.

A.5 Exponent bias

The exponent bias is derived from the format-defining parameters using the formula in §3.1.

This differs from IEEE-754, where the exponent bias is defined in terms of $e_{\max}$, the exponent of the largest finite value, which is in turn derived from the IEEE-754 interchange format parameters k and p .

The maximum exponent $e_{\max}$ is

$$e_{\max} = \begin{cases} 2^{K-1} - 3 & \text{if } P = 1, \Sigma = \text{Unsigned}, \Delta = \text{Extended} \\ 2^{K-1} - 2 & \text{if } P = 1, \Sigma = \text{Unsigned}, \Delta = \text{Finite} \\ 2^{K-2} - 2 & \text{if } P = 1, \Sigma = \text{Signed}, \Delta = \text{Extended} \\ 2^{K-2} - 2 & \text{if } P = 2, \Sigma = \text{Unsigned}, \Delta = \text{Extended} \\ 2^{K-P-1} - 1 & \text{otherwise if } \Sigma = \text{Signed} \\ 2^{K-P} - 1 & \text{otherwise if } \Sigma = \text{Unsigned} \end{cases}$$

A consequence of these specifications is that the datum 1.0 encodes consistently to the midway code point, that is 2^{K-2} for signed formats and 2^{K-1} for unsigned formats.

A.6 Subnormals

Subnormal numbers extend the dynamic range of floating-point values and induce equal quantization steps close to zero. This is useful when training models, where it is common for gradients to have near-zero non-zero values. Subnormals can also be useful to represent random values drawn from certain distributions, e.g., where model weights are initialized to small random values for training.

Subnormals are uniformly spaced around zero, and near-zero values are more probable under bell-shaped distributions. Formats with narrow exponent bitwidths necessarily have a limited range; subnormals extend this range by a power of two for every bit in the trailing significand.

The datum sets of formats with precision greater than one include subnormals; this does not preclude implementations of operations from flushing subnormals to zero, under the provision for approximate operations in §4.4.

Annex B

(informative)

Encoding

A K -bit floating-point value is encoded by an integer in the range 0 to $2^K - 1$. Some properties of this encoding are summarized here, while the detailed specification of the encoding is presented in §4.7.6.

All formats contain a single zero, encoded by the integer 0 .

All formats contain a single NaN. For signed formats, NaN is encoded at the code point which IEEE-754 uses for negative zero, that is 2^{K-1} . For unsigned formats, NaN is encoded at $2^K - 1$.

Extended formats contain one or two infinities. For signed formats, Inf is encoded at $2^{K-1} - 1$ and -Inf is encoded at $2^K - 1$. For unsigned formats, Inf is encoded at $2^K - 2$.

Table 3 summarizes the encodings of selected special values.

Datum	Symbol	Signed extended	Signed finite	Unsigned extended	Unsigned finite
Zero	0.0	0	0	0	0
One	1.0	$2^{K-2} - 0$	$2^{K-2} - 0$	$2^{K-1} - 0$	$2^{K-1} - 0$
Not a Number	NaN	$2^{K-1} - 0$	$2^{K-1} - 0$	$2^{K-0} - 1$	$2^{K-0} - 1$
Positive Infinity	Inf	$2^{K-1} - 1$	N/A	$2^{K-0} - 2$	N/A
Negative Infinity	-Inf	$2^{K-0} - 1$	N/A	N/A	N/A

Table 3—Encodings of selected datums for given K , independent of P .

B.1 Value tables

Tables 4, 5, 6, and 7 show the complete value sets for $K = 4$. Subnormals are shown in *underlined italic*, special values are shown in **bold**.

Code point	Binary4p1se	Binary4p2se	Binary4p3se
0 0b0000	Zero	Zero	Zero
1 0b0001	0.1250	<i><u>0.2500</u></i>	<i><u>0.2500</u></i>
2 0b0010	0.2500	0.5000	<i><u>0.5000</u></i>
3 0b0011	0.5000	0.7500	<i><u>0.7500</u></i>
4 0b0100	1.0000	1.0000	1.0000
5 0b0101	2.0000	1.5000	1.2500
6 0b0110	4.0000	2.0000	1.5000
7 0b0111	Inf	Inf	Inf
8 0b1000	NaN	NaN	NaN
9 0b1001	-0.1250	<i><u>-0.2500</u></i>	<i><u>-0.2500</u></i>
10 0b1010	-0.2500	-0.5000	<i><u>-0.5000</u></i>
11 0b1011	-0.5000	-0.7500	<i><u>-0.7500</u></i>
12 0b1100	-1.0000	-1.0000	-1.0000
13 0b1101	-2.0000	-1.5000	-1.2500
14 0b1110	-4.0000	-2.0000	-1.5000
15 0b1111	-Inf	-Inf	-Inf

Table 4—Value sets for $K = 4$, signed formats with $1 \leq P < 4$, extended domain.

Code point	Binary4p1sf	Binary4p2sf	Binary4p3sf
0 0b0000	Zero	Zero	Zero
1 0b0001	0.1250	<i><u>0.2500</u></i>	<i><u>0.2500</u></i>
2 0b0010	0.2500	0.5000	<i><u>0.5000</u></i>
3 0b0011	0.5000	0.7500	<i><u>0.7500</u></i>
4 0b0100	1.0000	1.0000	1.0000
5 0b0101	2.0000	1.5000	1.2500
6 0b0110	4.0000	2.0000	1.5000
7 0b0111	8.0000	3.0000	1.7500
8 0b1000	NaN	NaN	NaN
9 0b1001	-0.1250	<i><u>-0.2500</u></i>	<i><u>-0.2500</u></i>
10 0b1010	-0.2500	-0.5000	<i><u>-0.5000</u></i>
11 0b1011	-0.5000	-0.7500	<i><u>-0.7500</u></i>
12 0b1100	-1.0000	-1.0000	-1.0000
13 0b1101	-2.0000	-1.5000	-1.2500
14 0b1110	-4.0000	-2.0000	-1.5000
15 0b1111	-8.0000	-3.0000	-1.7500

Table 5—Value sets for $K = 4$, signed formats with $1 \leq P < 4$, finite domain.

Code point	Binary4p1ue	Binary4p2ue	Binary4p3ue	Binary4p4ue
0 0b0000	Zero	Zero	Zero	Zero
1 0b0001	≈ 0.0078	<u>0.0625</u>	<u>0.1250</u>	<u>0.1250</u>
2 0b0010	≈ 0.0156	0.1250	<u>0.2500</u>	<u>0.2500</u>
3 0b0011	≈ 0.0312	0.1875	<u>0.3750</u>	<u>0.3750</u>
4 0b0100	0.0625	0.2500	0.5000	<u>0.5000</u>
5 0b0101	0.1250	0.3750	0.6250	<u>0.6250</u>
6 0b0110	0.2500	0.5000	0.7500	<u>0.7500</u>
7 0b0111	0.5000	0.7500	0.8750	<u>0.8750</u>
8 0b1000	1.0000	1.0000	1.0000	1.0000
9 0b1001	2.0000	1.5000	1.2500	1.1250
10 0b1010	4.0000	2.0000	1.5000	1.2500
11 0b1011	8.0000	3.0000	1.7500	1.3750
12 0b1100	16.0000	4.0000	2.0000	1.5000
13 0b1101	32.0000	6.0000	2.5000	1.6250
14 0b1110	Inf	Inf	Inf	Inf
15 0b1111	NaN	NaN	NaN	NaN

Table 6—Value sets for $K = 4$, unsigned formats with $1 \leq P \leq 4$, extended domain.

Code point	Binary4p1uf	Binary4p2uf	Binary4p3uf	Binary4p4uf
0 0b0000	Zero	Zero	Zero	Zero
1 0b0001	≈ 0.0078	<u>0.0625</u>	<u>0.1250</u>	<u>0.1250</u>
2 0b0010	≈ 0.0156	0.1250	<u>0.2500</u>	<u>0.2500</u>
3 0b0011	≈ 0.0312	0.1875	<u>0.3750</u>	<u>0.3750</u>
4 0b0100	0.0625	0.2500	0.5000	<u>0.5000</u>
5 0b0101	0.1250	0.3750	0.6250	<u>0.6250</u>
6 0b0110	0.2500	0.5000	0.7500	<u>0.7500</u>
7 0b0111	0.5000	0.7500	0.8750	<u>0.8750</u>
8 0b1000	1.0000	1.0000	1.0000	1.0000
9 0b1001	2.0000	1.5000	1.2500	1.1250
10 0b1010	4.0000	2.0000	1.5000	1.2500
11 0b1011	8.0000	3.0000	1.7500	1.3750
12 0b1100	16.0000	4.0000	2.0000	1.5000
13 0b1101	32.0000	6.0000	2.5000	1.6250
14 0b1110	64.0000	8.0000	3.0000	1.7500
15 0b1111	NaN	NaN	NaN	NaN

Table 7—Value sets for $K = 4$, unsigned formats with $1 \leq P \leq 4$, finite domain.

Annex C (informative)

Value tables for K=2

Table 8 shows the floating-point value sets for $K = 2$ formats which follow the patterns in this document. Not all of these formats will typically be useful in practice, but they are included for completeness. As the table shows, certain peculiarities are evident:

- The formats Binary2p1ue and Binary2p2ue have the same datum set, albeit a different partitioning of subnormals.
- Similarly the formats Binary2p1uf and Binary2p2uf have the same datum set.
- The datum 1.0 is not present in Binary2p1se, Binary2p1ue, Binary2p2ue, hence is not encoded according to Table 3.
- Binary2p1se has no normal or subnormal values, hence $\text{MinPositiveOf}(\text{Binary2p1se}) = \text{Inf}$.
- Binary2p1se and Binary2p2ue have no normal values, hence $\text{MinNormalOf}(f) = \text{NaN}$ for these formats.

Code point	Binary2p1se	Binary2p1sf	Binary2p1ue	Binary2p2ue	Binary2p1uf	Binary2p2uf
0 0b00	Zero	Zero	Zero	Zero	Zero	Zero
1 0b01	Inf	1.000	0.500	<u>0.500</u>	0.500	<u>0.500</u>
2 0b10	NaN	NaN	Inf	Inf	1.000	1.000
3 0b11	-Inf	-1.000	NaN	NaN	NaN	NaN

Table 8—Value sets for $K = 2$. Subnormals are shown in underlined italic, special values in **bold**.

Annex D

(informative)

Examples of approximation declarations

D.1 Example: Approximate implementation of Exp

Consider an implementation of $\text{Exp}\langle \text{binary32}, \text{Binary8p4se}, (\text{NearestTiesToEven}, \text{SatNone}) \rangle$ which flushes subnormals in the result to zero or to the smallest normal, whichever is nearest, with ties going to zero. There are seven nonzero positive subnormals in Binary8p4se, of which four will be flushed to zero, and three will be returned as the smallest normal. Hence the value of κ over all inputs producing finite results is 4. Let I be the set of inputs for which the defined result is finite. Writing the cut points $a, b, c = \ln(2^{-11}), \ln(9 \cdot 2^{-11}), \ln(15 \cdot 2^{-11})$, the subsets are:

$$\begin{aligned} I_4 &= \mathcal{D}_{\text{binary32}} \cap (a, b) & \kappa_{I_4} &= 4 \\ I_3 &= \mathcal{D}_{\text{binary32}} \cap (b, c) & \kappa_{I_3} &= 3 \\ I_0 &= I \setminus (I_3 \cup I_4) & \kappa_{I_0} &= 0 \end{aligned}$$

and the implementation would declare

$$\kappa \in \{I_0 : 0, I_3 : 3, I_4 : 4\}.$$

Note that as a, b, c are irrational, the sets I_0, I_3, I_4 are disjoint and cover all inputs producing finite results.

D.2 Example: Approximate implementation of BlockDotProduct

Consider an implementation of dot product on blocks of size 8, with Binary8p4se elements and Binary8p1uf scale factors, with a binary32 result, rounding mode NearestTiesToEven and saturation mode SatNone. This is the operation specialization

$\text{BlockDotProduct}\langle 8, \text{Binary8p1uf}, \text{Binary8p4se}, \text{Binary8p1uf}, \text{Binary8p4se}, \text{binary32}, (\text{NearestTiesToEven}, \text{SatNone}) \rangle$

Suppose the nature of the implementation is such that it matches on NaNs and infinities, but has only the trivial bound $\kappa = 2^{32} - 2^{24} - 1$ over all inputs producing finite results. The implementation may choose to declare κ over a subset of the input space, for example by declaring

$$\begin{aligned} I_0 &= \{(s_x, [x_1, \dots, x_8], s_y, [y_1, \dots, y_8]) \text{ such that} \\ &\quad s_x = 1.0, \\ &\quad s_y = 1.0, \\ &\quad \text{Abs}(x_i) \leq 2.0 \text{ for } i \in \{1, 2\}, \\ &\quad x_i = 0.0 \text{ for } i \in \{3 \dots 8\}, \\ &\quad \text{Abs}(y_i) \leq 2.0 \text{ for } i \in \{1, 2\}, \\ &\quad y_i = 0.0 \text{ for } i \in \{3 \dots 8\} \\ &\quad \} \end{aligned}$$

and that $\kappa_{I_0} = 3$, while $\kappa_{I \setminus I_0}$ is the trivial bound as above.

This permits the implementation to provide a useful guarantee over a subset of the input space, while still providing a trivial guarantee over all inputs as required. Annex E shows an approach to characterization of accuracy for reduction operations.

Annex E

(informative)

Recommendations for reduction accuracy specifications

E.1 BlockReduceAdd

It is suggested that a finite error bound for the operation BlockReduceAdd be provided. Such a bound should apply at least whenever the output is finite. Examples of such bounds are given by Higham [6, §4.2] and by Blanchard et al. [1].

Example:

Consider a block of length $B = 8$, with arguments and output in Binary8p4se, and scale factor 1, where the accumulation is performed in binary16 using NearestTiesToEven, and the inputs are summed sequentially in any order. Let

$$S = X_1 + \cdots + X_8$$

denote the true sum, and let $\tilde{S}$ denote the implementation's approximate result. In the absence of overflow in $\tilde{S}$,

$$|S - \tilde{S}| \leq |S|/16 + 3.637 \times 10^{-3} \sum_{i=1}^8 |X_i|.$$

The constant 3.637×10^{-3} is obtained by rounding up $(1 + 1/16)((1 + 2^{-11})^7 - 1)$, where the constants are obtained as follows: 11 is the accumulator precision, i.e., PrecisionOf(binary16); $7 = B - 1$ is the number of additions; and $16 = 2^{\text{PrecisionOf(Binary8p4se)}}$.

E.2 BlockReduceMultiply

It is suggested that a finite error bound for the operation BlockReduceMultiply be provided. Such a bound should apply at least whenever the output is finite. Examples of such bounds are given by Higham [6, §2.2].

Example:

Consider a block of length 8, with arguments and output in Binary8p4se, and scale factor 1, where the multiplications are performed in binary16 using NearestTiesToEven, sequentially in any order. Let

$$P = X_1 \times \cdots \times X_8$$

denote the true product, and let $\tilde{P}$ denote the implementation's approximate result. In the absence of underflow or overflow, both intermediate and final,

$$|P - \tilde{P}| \leq |P| \times 6.614 \times 10^{-2}.$$

The constant 6.614×10^{-2} is obtained by rounding up $(1 + 1/16)(1 + 2^{-11})^7 - 1$, where the constants are obtained as follows: 11 is the accumulator precision, i.e., PrecisionOf(binary16); $7 = B - 1$ is the number of multiplications; and $16 = 2^{\text{PrecisionOf(Binary8p4se)}}$.

E.3 BlockDotProduct

It is suggested that a finite error bound for the operation BlockDotProduct be provided. Such a bound should apply at least whenever the output is finite and the final rounding does not underflow. Examples of such bounds are given by Higham [6, §3.1] and by Blanchard et al. [1]. More generally applicable bounds may be complicated by implementation choices, for example when the precision used for the individual products varies during the computation.

Example:

Consider a block of length 8, with arguments and output in Binary8p4se, and scale factors 1, where the multiplications and accumulation are performed in binary16 using NearestTiesToEven, and the products are summed sequentially in any order. Let

$$D = X_1 \times Y_1 + \cdots + X_8 \times Y_8$$

denote the true dot product, and let $\tilde{D}$ denote the implementation's approximate result. In the absence of overflow, both intermediate and final, and if the final rounding does not underflow, then

$$|D - \tilde{D}| \leq |D|/16 + 3.637 \times 10^{-3} \sum_{i=1}^8 |X_i \times Y_i|.$$

In this example the products $X_i \times Y_i$ are exact, while intermediate underflows, if they occur, are exact.

The constant 3.637×10^{-3} is obtained by rounding up $(1 + 1/16)((1 + 2^{-11})^7 - 1)$, where the constants are obtained as follows: 11 is the accumulator precision, i.e., PrecisionOf(binary16); $7 = B - 1$ is the number of additions; and $16 = 2^{\text{PrecisionOf(Binary8p4se)}}$.

Annex F

(informative)

Operation groups

Operations are defined in groups as follows

Group A (Core)

Convert $\langle f_x, f_r, \rho \rangle$
Negate $\langle f_x, f_r, \rho \rangle$
Abs $\langle f_x, f_r, \rho \rangle$
Add $\langle f_x, f_y, f_r, \rho \rangle$
Subtract $\langle f_x, f_y, f_r, \rho \rangle$
Multiply $\langle f_x, f_y, f_r, \rho \rangle$
FMA $\langle f_x, f_y, f_z, f_r, \rho \rangle$
FAA $\langle f_x, f_y, f_z, f_r, \rho \rangle$
Minimum $\langle f_x, f_y, f_r, \rho \rangle$
Maximum $\langle f_x, f_y, f_r, \rho \rangle$
MinimumNumber $\langle f_x, f_y, f_r, \rho \rangle$
MaximumNumber $\langle f_x, f_y, f_r, \rho \rangle$
MinimumMagnitude $\langle f_x, f_y, f_r, \rho \rangle$
MaximumMagnitude $\langle f_x, f_y, f_r, \rho \rangle$
MinimumMagnitudeNumber $\langle f_x, f_y, f_r, \rho \rangle$
MaximumMagnitudeNumber $\langle f_x, f_y, f_r, \rho \rangle$
MinimumFinite $\langle f_x, f_y, f_r, \rho \rangle$
MaximumFinite $\langle f_x, f_y, f_r, \rho \rangle$
Clamp $\langle f_x, f_{lo}, f_{hi}, f_r, \rho \rangle$
CompareLess $\langle f_x, f_y \rangle$
CompareLessEqual $\langle f_x, f_y \rangle$
CompareEqual $\langle f_x, f_y \rangle$
CompareGreater $\langle f_x, f_y \rangle$
CompareGreaterEqual $\langle f_x, f_y \rangle$
IsZero $\langle f \rangle$
IsOne $\langle f \rangle$
IsNaN $\langle f \rangle$
IsInfinite $\langle f \rangle$
IsFinite $\langle f \rangle$
IsSignMinus $\langle f \rangle$
IsNormal $\langle f \rangle$
IsSubnormal $\langle f \rangle$

Group KA (Block Core)

ConvertFromBlock $\langle B, f_s, f_x, f_r, \rho \rangle$
ConvertToBlock $\langle B, f_x, f_s, f_r, \rho \rangle$
ConvertToBlockMaxAbsFinite $\langle B, f_x, f_s, f_r, \rho_s, \rho \rangle$
BlockConvert $\langle B, f_{s1}, f_{x1}, f_s, f_r, \rho \rangle$
BlockNegate $\langle B, f_{s1}, f_{x1}, f_s, f_r, \rho \rangle$
BlockAbs $\langle B, f_{s1}, f_{x1}, f_s, f_r, \rho \rangle$
BlockAdd $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockSubtract $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockMultiply $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockFMA $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_{s3}, f_{x3}, f_s, f_r, \rho \rangle$
BlockFAA $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_{s3}, f_{x3}, f_s, f_r, \rho \rangle$
BlockMinimum $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockMaximum $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockMinimumNumber $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockMaximumNumber $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockMinimumMagnitude $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockMaximumMagnitude $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockMinimumMagnitudeNumber $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockMaximumMagnitudeNumber $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockMinimumFinite $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockMaximumFinite $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho \rangle$
BlockClamp $\langle B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_{s3}, f_{x3}, f_s, f_r, \rho \rangle$

Group M (Meta)

BitwidthOf $\langle f \rangle$
PrecisionOf $\langle f \rangle$
SignednessOf $\langle f \rangle$
DomainOf $\langle f \rangle$
ExponentBitwidthOf $\langle f \rangle$
TrailingSignificandBitwidthOf $\langle f \rangle$
ExponentBiasOf $\langle f \rangle$
MaxFiniteOf $\langle f \rangle$
MinFiniteOf $\langle f \rangle$
MinPositiveOf $\langle f \rangle$
MaxSubnormalOf $\langle f \rangle$
MinNormalOf $\langle f \rangle$

Group B (Basic)

Divide< f_x, f_y, f_r, ρ >
Sqrt< f_x, f_r, ρ >
Recip< f_x, f_r, ρ >
RSqrt< f_x, f_r, ρ >
Exp< f_x, f_r, ρ >
Exp2< f_x, f_r, ρ >
ExpMinusOne< f_x, f_r, ρ >
Log< f_x, f_r, ρ >
Log2< f_x, f_r, ρ >
LogOnePlus< f_x, f_r, ρ >
Softplus< f_x, f_r, ρ >
NextGreaterThan< f >
NextLessThan< f >
CopySign< f_x, f_y, f_r, ρ >
TotalOrder< f_x, f_y >
Class< f >

Group C (Full)

Hypot< f_x, f_y, f_r, ρ >
ArcTan2< f_y, f_x, f_r, ρ >
ArcTan2Pi< f_y, f_x, f_r, ρ >
Sin< f_x, f_r, ρ >
Cos< f_x, f_r, ρ >
Tan< f_x, f_r, ρ >
ArcSin< f_x, f_r, ρ >
ArcCos< f_x, f_r, ρ >
ArcTan< f_x, f_r, ρ >
Sinh< f_x, f_r, ρ >
Cosh< f_x, f_r, ρ >
Tanh< f_x, f_r, ρ >
ArcSinh< f_x, f_r, ρ >
ArcCosh< f_x, f_r, ρ >
ArcTanh< f_x, f_r, ρ >
SinPi< f_x, f_r, ρ >
CosPi< f_x, f_r, ρ >
TanPi< f_x, f_r, ρ >
ArcSinPi< f_x, f_r, ρ >
ArcCosPi< f_x, f_r, ρ >
ArcTanPi< f_x, f_r, ρ >

Group KB (Block Basic)

BlockDivide< $B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho$ >
BlockSqrt< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockRecip< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockRSqrt< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockExp< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockExp2< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockExpMinusOne< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockLog< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockLog2< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockLogOnePlus< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockSoftplus< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockCopySign< $B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho$ >

Group KC (Block Full)

BlockHypot< $B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho$ >
BlockArcTan2< $B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho$ >
BlockArcTan2Pi< $B, f_{s1}, f_{x1}, f_{s2}, f_{x2}, f_s, f_r, \rho$ >
BlockSin< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockCos< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockTan< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockArcSin< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockArcCos< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockArcTan< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockSinh< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockCosh< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockTanh< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockArcSinh< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockArcCosh< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockArcTanh< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockSinPi< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockCosPi< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockTanPi< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockArcSinPi< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockArcCosPi< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >
BlockArcTanPi< $B, f_{s1}, f_{x1}, f_s, f_r, \rho$ >

Group E (Block Reductions)

BlockReduceAdd< B, f_s, f_x, f_r, ρ >
BlockReduceMultiply< B, f_s, f_x, f_r, ρ >
BlockDotProduct< $B, f_{sx}, f_x, f_{sy}, f_y, f_r, \rho$ >

Group SA (Scaled Core)

ScaledConvert $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledNegate $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledAbs $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledAdd $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledSubtract $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledMultiply $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledFMA $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), (f_{s3}, f_{x3}), f_r, \rho \rangle$
 ScaledFAA $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), (f_{s3}, f_{x3}), f_r, \rho \rangle$
 ScaledMinimum $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledMaximum $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledMinimumNumber $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledMaximumNumber $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledMinimumMagnitude $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledMaximumMagnitude $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledMinimumMagnitudeNumber $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledMaximumMagnitudeNumber $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledMinimumFinite $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledMaximumFinite $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledClamp $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), (f_{s3}, f_{x3}), f_r, \rho \rangle$

Group SB (Scaled Basic)

ScaledDivide $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledSqrt $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledRecip $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledRSqrt $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledExp $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledExp2 $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledExpMinusOne $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledLog $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledLog2 $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledLogOnePlus $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledSoftplus $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledCopySign $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$

Group SC (Scaled Full)

ScaledHypot $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledArcTan2 $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledArcTan2Pi $\langle (f_{s1}, f_{x1}), (f_{s2}, f_{x2}), f_r, \rho \rangle$
 ScaledSin $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledCos $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledTan $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledArcSin $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledArcCos $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledArcTan $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledSinh $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledCosh $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledTanh $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledArcSinh $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledArcCosh $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledArcTanh $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledSinPi $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledCosPi $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledTanPi $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledArcSinPi $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledArcCosPi $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$
 ScaledArcTanPi $\langle (f_{s1}, f_{x1}), f_r, \rho \rangle$

Annex G

(informative)

Bibliography

References

- [1] P. Blanchard, N. J. Higham, F. Lopez, T. Mary, and S. Pranesh, “Mixed precision block fused multiply-add: Error analysis and application to GPU tensor cores,” *SIAM Journal on Scientific Computing*, vol. 42, no. 3, pp. C124–C141, 2020.
- [2] T.-C. Chang, S. Park, J. P. Lim, and S. Nagarakatte, “FLoPS: Semantics, operations, and properties of P3109 floating-point representations in Lean,” 2026, Rutgers Department of Computer Science Technical Report DCS-TR-762. [Online]. Available: <https://arxiv.org/abs/2602.15965>
- [3] A. W. Fitzgibbon and S. Felix, “On stochastic rounding with few random bits,” in *IEEE 32nd Symposium on Computer Arithmetic, ARITH 2025, El Paso, TX, USA, May 4-7, 2025*. IEEE, 2025, pp. 133–140. [Online]. Available: <https://doi.org/10.1109/ARITH64983.2025.00029>
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, ch. 6.2.2.3 Softmax Units for Multinoulli Output Distributions, pp. 180–184.
- [5] Google, “Jax lax package: `_float_to_int_for_sort`,” https://github.com/google/jax/blob/fc5960f2b8/jax/_src/lax/lax.py#L3934.
- [6] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM, 2002.
- [7] IEEE Computer Society, “IEEE standard for floating-point arithmetic,” *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, 2019.
- [8] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, “Cloud TPU: Machine learning accelerators for training and inference,” *IEEE Micro*, vol. 38, no. 2, pp. 39–47, 2018.
- [9] W. Kahan, “Branch cuts for complex elementary functions or much ado about nothing’s sign bit,” *Institute of Mathematics and its Applications Conference*, 1987, https://www.arithmazium.org/library/lib/wk_branch_cuts_86.pdf.
- [10] W. Kahan and J. W. Thomas, “Augmenting a programming language with complex arithmetic,” EECS Department, University of California, Berkeley, Tech. Rep. UCB/CSD-92-667, 1991, <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1992/6127.html>.
- [11] PyTorch authors, “Pytorch torchtext package: `_t5_multi_head_attention_forward`,” <https://github.com/pytorch/text/blob/a933cbe5a008bc2cb61d985cf5864069194157eb/torchtext/prototype/models/t5/modules.py#L236>.
- [12] C. M. Wintersteiger, “Formal verification of the IEEE P3109 standard for binary floating-point formats for machine learning,” in *IEEE 32nd Symposium on Computer Arithmetic, ARITH 2025, El Paso, TX, USA, May 4-7*. IEEE, 2025, <https://github.com/imandra-ai/ieee-p3109>.
- [13] R. Zhao, B. Vogel, and T. Ahmed, “Adaptive loss scaling for mixed precision training,” arXiv cs.LG, Tech. Rep., 2019, <https://arxiv.org/abs/1910.12385>.